SG DMA — SFR (레지스터) 맵

항목	내용
문서 ID	SGDMA-SFR-rel-v2.0.0
작성일	2026-10-16 (v2.0.0 개정) · 2026-10-06 (v1.0.0 인계)
작성자	hrcho (v1.0.0)
대상 DUT	**`rtl/sg_dma/sgdma_top.v`** (ADI `axi_dmac` 기반 SG DMA)
DUT 버전	`VERSION` = `0x0004_0565` (4.05.65)
HDL 출처	`analogdevicesinc/hdl` `hdl_2026_r1` / `b62abeff`
기준 구성	AXI-MM → AXI-MM / 단일 클록 / SG 활성 / 2D 비활성 / **`DMA_AXI_ADDR_WIDTH = 32`** / **`AXI_AXCACHE = 4'b0000`**
검증 레벨	IP DV (block-level)
상태	**검증팀 인계본** — v2.0.0은 **문서만** 고쳤습니다. 레지스터 주소·비트·접근 유형·리셋값·근거 등급·RTL 위치는 v1.0.0과 같습니다
원본문서	$repo/docs/ref_hrcho/261006-claude-sgdma_sfr_map-001-hrcho.md

관련 문서

문서	역할
`SGDMA-SOR-rel-v2.0.0`	**SOR (검증 요구사항)** — 테스트 15개(T-01 ~ T-15). 원래 요구사항 61건 중 58건을 흡수, 3건 제외. 이 문서의 SOR 연결은 모두 이 판 기준 → §9.4
`SGDMA-SANITY-rel-v2.0.0`	**Sanity Test** — 회귀 게이트 46 checks. 본 문서에서 근거 **M**인 값의 출처
`SGDMA-SOR-INTEGRATION-v1.0`	SOR 축소안 통합 문서. 7장 "SFR 수정 4~8"이 이번 개정의 근거
`$repo/docs/ref_hrcho/260930-claude-rtl_design_crosscheck-hrcho.md`	RTL 구성 교차 검증 (0-replication 패치 근거)

> **절 번호는 v1.0.0과 같습니다.** SANITY v2.0.0 부록 B와 SOR v2.0.0이 이 문서의 절 번호(예: §4.6.2, §4.7.3, §5, §8)를 참조하기 때문입니다. v2.0.0에서 새로 넣은 내용은 기존 절 안에 넣거나, 절 끝에 새 하위 절(예: §6.4, §9.4)로 붙였습니다.

이 문서를 처음 읽는 분께

비유로 이해하기

레지스터는 DMA의 조작 패널입니다. CPU는 이 패널의 버튼을 누르고 표시등을 읽어서 DMA를 다룹니다.

- **버튼** — `CONTROL`(켜기·일시정지), `TRANSFER_SUBMIT`(출발 버튼)처럼 "누르면 일이 일어나는" 레지스터.
- **스위치** — `IRQ_MASK`처럼 "어떤 알림을 받을지" 정해 두는 레지스터.
- **표시등** — `TRANSFER_DONE`, `IRQ_PENDING`처럼 "지금 상태"를 보여 주는 레지스터.
- **명판** — `VERSION`, `IDENTIFICATION`처럼 제품 이름과 버전이 새겨진 고정값 레지스터.

그런데 SG DMA의 진짜 할 일은 패널에 있지 않습니다. CPU가 메모리에 **작업 지시서 묶음**(descriptor 체인)을 적어 두고, 패널에는 **첫 장이 어디 있는지만** 알려 줍니다(`SG_ADDRESS`). 그래서 이 문서는 패널(1~4장)과 지시서 양식(5장)을 함께 설명합니다.

SG DMA 자체가 처음이라면 `SGDMA-SOR-rel-v2.0.0` 1장("SG DMA 처음 보는 분께")을 먼저 읽으십시오.

먼저 볼 네 곳

전체 40여 개 레지스터를 순서대로 읽기보다, **먼저 아래 네 곳을 보십시오.** 이 네 가지를 모르면 정상 동작을 버그로 보고하거나 그 반대가 됩니다.

#	위치	내용
1	§ 2.5 근거 등급	각 값이 **"돌려서 확인한 것(M)"인지 "RTL을 읽어서 추론한 것(R)"인지**를 모든 항목에 표기했습니다. 통합 과정에서 R이 틀렸던 사례가 실제로 있었습니다 (§ 4.1.6).
2	부록 A	리셋 직후 readback 기대값 전체. **비영(非零, 0이 아닌) 리셋값은 `FLAGS = 0x2` 와 `IRQ_MASK = 0x3` 두 개뿐**입니다. "리셋 후 전 레지스터 0" 가정 테스트는 이 둘에서만 FAIL합니다.
3	§ 4.8	파라미터로 **의도적으로 죽인 비트** 목록. 기대값을 모르면 "RW 레지스터 write/read-back" 테스트가 **정상 동작을 버그로 보고**합니다.
4	§ 4.5.5	**`0x450` 은 읽으면 내부 FIFO가 pop됩니다(하나 빠집니다).** "전 레지스터 덤프" 방식 테스트를 만들면 안 되는 이유입니다.

SG 고유 레지스터는 `SG_ADDRESS` (0x47C)와 `DESCRIPTOR_ID` (0x454) 두 개이며, SG 동작의 실체는 레지스터가 아니라 **메모리상의 descriptor 포맷**(§ 5)에 있습니다.

> ⚠️ **꼭 기억할 규칙 세 가지 (v2.0.0에서 한 곳으로 모음)**
>
> 1. **디버그 레지스터(`0x434` ~ `0x444`)는 관찰용입니다.** 값을 기록만 하고 합격 기준에 쓰지 않습니다 → § 4.7.
> 2. **`0x450` 은 어떤 이유로도 읽지 않습니다** → § 4.5.5.
> 3. **완료 판정은 EOT 알림 또는 `TRANSFER_DONE` 으로만** 합니다. `DESCRIPTOR_ID` 나 "접수(`TRANSFER_QUEUED`)" 알림으로 판정하지 않습니다 → § 4.6.2, § 6.4.

처음 보는 말 풀이

말	쉬운 뜻
SFR	Special Function Register. CPU가 IP를 조작하는 레지스터. 이 문서의 대상
DUT	검증하는 회로. 여기서는 `sgdma_top`
RTL	회로를 글로 적은 설계 코드 (Verilog)
Offset	레지스터의 **바이트 주소**. 드라이버·TB가 쓰는 값 (예: `0x400`)
Word	RTL `case` 문에 쓰인 **9비트 word 주소**. Offset ÷ 4 (§ 2.1)
비트 필드	32비트 레지스터 안에서 한 가지 뜻을 가진 비트 묶음 (예: `CONTROL[2]` = `HWDESC`)
리셋값	전원을 넣거나 리셋을 풀었을 때 레지스터에 처음 들어 있는 값
RO / RW / RW1C …	접근 유형. 읽기만 되는지, 쓰면 바뀌는지, 1을 쓰면 지워지는지 (§ 2.2)
근거 등급 M / R / D	값의 출처. M = 시뮬레이션으로 실측, R = RTL을 읽고 계산(미측정), D = upstream 문서 인용 (§ 2.5)
파라미터	합성할 때 정하는 설정값 (예: 데이터 폭 64). 바꾸면 레지스터 모양도 바뀜
`generate`	파라미터에 따라 회로를 **아예 만들지 않을 수 있는** Verilog 문법
upstream	원본 ADI `axi_dmac` 코드와 그 공식 문서
descriptor	전송 1건의 작업 지시서 (메모리에 놓인 48바이트). § 5
체인	descriptor가 "다음 장 주소"로 줄줄이 이어진 것
HWDESC	`CONTROL[2]` . 1이면 SG 모드(지시서를 메모리에서 읽음), 0이면 레지스터 모드
SOT / EOT	전송 접수(Start Of Transfer) / 전송 완료(End Of Transfer) 사건. 알림 비트0 / 비트1
W1C	Write-1-to-Clear. 1을 쓰면 지워지는 비트
FIFO / pop	먼저 들어온 것이 먼저 나가는 대기열 / 대기열에서 하나를 꺼내는 것
beat / burst	버스의 한 박자(이 DUT는 8바이트) / 주소 한 번으로 여러 박자를 묶은 덩어리(최대 128바이트)

말	쉬운 뜻
AXI4-Lite	CPU가 레지스터를 읽고 쓰는 간단한 버스 (`s_axi` 포트)
Gray 코드	이웃한 값끼리 비트가 하나만 다른 숫자 표기법. 디버그 ID에 쓰임 (부록 B)
Inactive / Unreachable	파라미터로 꺼진 레지스터 / 동작은 하지만 이 구성에서는 값을 바꿀 일이 없는 레지스터 (§2.4)
hang	멈춰서 끝나지 않는 상태. 이 문서와 SOR에서의 정확한 뜻은 SOR v2.0.0 2.3절 (§5 주의 2)
T-xx	SOR v2.0.0의 테스트 번호 (T-01 ~ T-15)
SAN-xxx	Sanity Test의 TC 번호 (SAN-001 ~ SAN-007)

확정된 설계 결정 (2026-10-06)

본 문서는 아래 7건의 합의를 반영한 확정본입니다. 결정 전 논의 내역은 부록 D에 보존했습니다.

#	항목	결정	본 문서 반영
1	`AXI_AXCACHE`	**`4'b0000` 유지 + 래퍼에 명시**	§1.1, §4.1.6
2	`DMA_AXI_ADDR_WIDTH`	**`32` 확정**	전 문서. 64 전환 영향은 §10에 분리
3	모듈명	**`sgdma_top`으로 통일**	전 문서
4	래퍼 파라미터	**전부 명시** (기본값과 같아도. 결정 당시 36개, `AXI_AXCACHE`·`AXI_AXPROT` 추가 후 38개)	§1.1 — "미지정 / 기본값" 행이 사라짐
5	AXI ID / LOCK 포트	**노출 유지**	레지스터 맵 영향 없음. §1.3
6	레지스터 명칭 4건	**upstream 명칭으로 통일**	§3 이하 전체. 코드 반영 결과는 부록 E
7	Sanity 보강 2건	**재조정** — Sanity는 46 checks 유지. 미측정 리셋값은 **SOR에서도 범위 밖**	§9, 부록 D.3

> **결정 1·2는 "값을 바꾸는 결정"이 아니라 "현재 값을 확정하는 결정"입니다.** 두 파라미터 모두 as-built(지금 만들어진 그대로) 값이 유지되므로, **기존 46개 체크의 기대값은 하나도 바뀌지 않습니다.** 변하는 것은 래퍼에 값이 명시되어 **의도가 코드에 남는다**는 점입니다.

v1.0.0 대비 바뀐 점

RTL의 사실은 하나도 바뀌지 않았습니다. 레지스터 주소, 비트 위치, 접근 유형, 리셋값, 근거 등급, 파라미터 값, Gray 코드 표, RTL 소스 위치, 결정 이력은 v1.0.0과 같습니다. 절 번호도 같습니다. 바뀐 것은 설명과 연결입니다.

#	바뀐 점	위치	근거
1	`x_len` / `X_LENGTH`는 "바이트 수 − 1"이고 `0` = 1바이트, `0xFFFFFF` = 16 MiB이며 **길이 0 descriptor는 만들 수 없다**는 점을 명확히 적음. v1.0.0 SOR ET-005의 "길이 0 = `0xFFFFFF`"가 틀렸다는 정정 메모 추가	§4.4.6, §5	통합 문서 7장 #4
2	자기 참조(순환) 체인 설명에 SOR v2.0.0 2.3절 **"hang 없음"** 정의(ENABLE=0 후 버스 idle 복귀 + 다시 켠 뒤 정상 전송)를 연결	§5 주의 2	통합 문서 7장 #5
3	디버그 레지스터(`0x434` ~ `0x444`)는 **관찰용이며 판정에 쓰지 않는다**는 규칙을 §4.7 서두 한 곳에 정리. `DBG_STATUS`를 "판정할 수 있다"고 적었던 문장을 "관찰 기록"으로 고침. `0x450` 읽기 금지 유지	§4.7, §4.7.3	통합 문서 7장 #6
4	요구사항 ID가 나오는 곳을 **SOR v2.0.0 테스트 번호(T-01 ~ T-15)** 기준으로 갱신. 원래 ID(`REG-xxx`, `FT-xxx` 등)는 괄호로 유지. SOR 대응표 §9.4 신설. §9.3을 SANITY v2.0.0 5장과 맞춤	§4 전반, §8, §9, §10, §11	통합 문서 7장 #7
5	`IRQ_MASK = 0x0`이면 출발 직후 **"접수(`TRANSFER_QUEUED`)" 알림이 먼저** 올라오므로 완료로 오인하지 말라는 주의를 추가	§4.2.1, §6.3, §6.4 신설, §7.2	통합 문서 7장 #8

#	바뀐 점	위치	근거
6	`DESCRIPTOR_ID` — 완료 후 값은 **구현 특성(계약 아님)**, 알림 시점에 읽으면 **다음 장 id가 보일 수 있음** (SOR T-08과 일치)	§4.6.2	통합 문서 5장 F-08
7	§8 영향 매트릭스가 **RTL 분석(미측정)** 이며 SOR T-11/T-12에서 **처음 측정**한다는 연결을 명시	§8	통합 문서 5장 F-12
8	문서 참조 이름을 v2.0.0으로 바꿈 (`SGDMA-SOR-rel-v2.0.0`, `SGDMA-SANITY-rel-v2.0.0`)	머리말, §9	—
9	처음 보는 사람도 읽을 수 있게 다시 씀: 조작 패널 비유, 용어 풀이, §3 "한 줄 설명" 열, 레지스터마다 "한 줄 요약"과 "⚠️ 주의"	전체	—

목차

- 1. 기준 구성
- 2. 레지스터 인터페이스 공통 사양
- 3. 레지스터 맵 요약
- 4. 레지스터 상세
- 5. SG 디스크립터 메모리 포맷
- 6. 인터럽트 동작
- 7. 프로그래밍 시퀀스
- 8. 리셋 · ENABLE · PAUSE 영향 매트릭스
- 9. Sanity Test 연계
- 10. ADDR_WIDTH = 64 전환 시 영향
- 11. 미확정 항목
- 부록 A — 리셋 직후 readback 기대값
- 부록 B — Gray 코드 변환표
- 부록 C — RTL 소스 위치
- 부록 D — 결정 및 충돌 해소 이력
- 부록 E — 코드 반영 현황

1. 기준 구성

이 장은 "어떤 설정으로 만든 DMA인가"를 적습니다. 같은 `axi_dmac` 코드라도 **파라미터가 다르면 레지스터 모양이 달라집니다.** 그래서 레지스터를 보기 전에 설정부터 고정합니다.

1.1 레지스터 맵에 영향을 주는 파라미터

`sgdma_top.v`가 `axi_dmac`에 넘기는 파라미터는 **38개**이며, 결정 4에 따라 **기본값과 같은 것도 전부 명시**되어 있습니다. 그중 레지스터의 **필드 폭 · 리셋값 · 쓰기 가능 여부**를 바꾸는 것만 추렸습니다.

> **2026-10-06 이전에는 36개였고, `AXI_AXCACHE`·`AXI_AXPROT`가 빠져 있었습니다.** 결정 1·4로 두 개를 추가해 38개가 되었습니다. 추가 전후로 **Sanity 46/46 및 AXI 트랜잭션 타임스탬프 19개가 전부 동일**함을 확인했습니다. 즉 동작은 그대로이고, 의도만 코드에 남긴 변경입니다.

파라미터	값	레지스터 맵 영향
`ID`	`0`	`PERIPHERAL_ID`(0x004) 읽기값
`DMA_TYPE_SRC` / `_DEST`	`0` / `0` (AXI-MM)	`INTERFACE_DESCRIPTION_1` 필드, `SRC/DEST_ADDRESS` 활성, `X_LENGTH` 정렬 제약 없음

파라미터	값	레지스터 맵 영향
`DMA_DATA_WIDTH_SRC` / `_DEST` / `_SG`	`64`	주소 레지스터 하위 3비트 무시
`DMA_AXI_ADDR_WIDTH`	**`32`**	**`*_ADDRESS_HIGH` 레지스터 전부 비활성**
`DMA_LENGTH_WIDTH`	`24`	`X_LENGTH` 유효 폭 24비트
`DMA_2D_TRANSFER`	`0`	`Y_LENGTH`, `*_STRIDE` 비활성
`DMA_SG_TRANSFER`	**`1`**	**`CONTROL.HWDESC` 쓰기 가능, `SG_ADDRESS·DESCRIPTOR_ID` 활성**
`CYCLIC`	`0`	`FLAGS.CYCLIC` 쓰기 무시 (항상 0)
`FRAMELOCK`	`0`	`CONTROL.FRAMELOCK` 쓰기 무시, `FRAMELOCK_*` 비활성
`MAX_NUM_FRAMES_WIDTH`	`3`	`INTERFACE_DESCRIPTION_1.MAX_NUM_FRAMES` = 7
`MAX_BYTES_PER_BURST`	`128`	`INTERFACE_DESCRIPTION_1.BYTES_PER_BURST_WIDTH` = 7
`FIFO_SIZE`	`8`	`ID_WIDTH` = 4 → `DBG_IDS0/1` 필드 폭
`USE_EXT_SYNC` / `AUTORUN`	`0` / `0`	`INTERFACE_DESCRIPTION_1` 상위 비트, 모든 리셋값이 non-AUTORUN 쪽
`CACHE_COHERENT`	`0`	`INTERFACE_DESCRIPTION_2[0]`
`AXI_AXCACHE`	**`4'b0000`**	**`INTERFACE_DESCRIPTION_2[7:4]`** — 결정 1로 래퍼에 명시 **(적용 완료)**
`AXI_AXPROT`	**`3'b000`**	**`INTERFACE_DESCRIPTION_2[10:8]`** — 결정 4로 래퍼에 명시 **(적용 완료)**
`DISABLE_DEBUG_REGISTERS`	`0`	디버그 레지스터 읽힘
`SYNC_TRANSFER_START`	`0`	레지스터 없음

> **결정 1·4로 해소된 위험.** rev.002까지 `AXI_AXCACHE`/`AXI_AXPROT`는 래퍼에 **미지정**이었고, `axi_dmac`의 기본값이 암묵적으로 적용되고 있었습니다. 그 결과 두 선행 문서가 모두 `INTERFACE_DESCRIPTION_2` 리셋값을 틀리게 계산했습니다(§ 4.1.6). **값 자체는 그대로지만, 명시되어 있으면 같은 오류가 반복되지 않습니다.** 이것이 "기본값과 같아도 전부 명시"(결정 4, 지금은 38개)의 실질적 이유입니다.

1.2 파생 localparam

localparam은 파라미터로부터 **자동으로 계산되는** 내부 상수입니다. 레지스터 비트 폭은 대부분 여기서 정해집니다.

localparam	계산	본 구성 값	쓰이는 곳
`BYTES_PER_BEAT_WIDTH_SRC/DEST/SG`	`log2(64/8)`	**3**	주소 레지스터 쓰기 가능 비트 `[31:3]`
`DMA_LENGTH_ALIGN`	SRC·DEST 모두 MM → `max(0,0)`	**0**	`X_LENGTH` 쓰기 가능 비트 `[23:0]`
`REAL_MAX_BYTES_PER_BURST`	`min(128, 256×64/8)`	**128**	burst 분할
`BYTES_PER_BURST_WIDTH`	`log2(128)`	**7**	`INTERFACE_DESCRIPTION_1[19:16]`
`ID_WIDTH`	`FIFO_SIZE > 4 ? 4`	**4**	`DBG_IDS0/1` 필드
`DBG_ID_PADDING`	`8 − ID_WIDTH`	**4**	`DBG_IDS0/1` 상위 패딩
`HAS_ADDR_HIGH`	`DMA_AXI_ADDR_WIDTH > 32`	**0**	`*_HIGH` 레지스터 **비활성**
`ADDR_LOW_MSB`	`HAS_ADDR_HIGH ? 31 : ADDR_WIDTH−1`	**31**	주소 레지스터 폭
`HAS_SRC_ADDR` / `HAS_DEST_ADDR`	`DMA_TYPE_* == 0`	**1 / 1**	주소 레지스터 읽기값 게이트
`MEASURED_LENGTH_WIDTH`	`DMA_2D_TRANSFER ? 32 : DMA_LENGTH_WIDTH`	**24**	`TRANSFER_PROGRESS` 폭
`MAX_NUM_FRAMES`	`2**3`	**8**	`INTERFACE_DESCRIPTION_1[31:27]` = 7
`PCORE_VERSION`	상수	**`0x0004_0565`**	`VERSION`

⚠️ **`DMA_LENGTH_ALIGN = 0`은 본 구성의 고유 특성입니다.** SRC와 DEST가 **둘 다 AXI-MM**일 때만 0이 됩니다. ADI 공식 testbench(`testbenches/ip/dma_sg`)는 두 config 모두 한쪽이 AXI-Stream이라 이 경로를 검증하지 않습니다. `X_LENGTH` 바이트 단위 지정(§4.4.6)과 `burst_memory.v` 0-replication 패치가 모두 여기에서 나옵니다. SOR v2.0.0에서는 **T-07**(1바이트 단위 길이 전송)이 이 경로를 검증합니다.

1.3 레지스터 맵에 영향이 없는 변경

결정 3·5는 레지스터 사양을 바꾸지 않습니다. 참고로만 기록합니다.

결정	변경 내용	레지스터 맵 영향
3	모듈명 `sg_dma_top` → `sgdma_top` **(적용 완료)**	**없음** (파일·인스턴스 이름)
5	`m_*_axi_a*id` / `a*lock` / `*id` 포트 노출, `AXI_ID_WIDTH = 1` **(적용 완료)**	**없음** — `INTERFACE_DESCRIPTION_*`에 ID 폭 필드가 존재하지 않음

결정 5가 동작을 바꾸지 않았다는 근거(behavior-neutral). 포트 추가 전후로 Sanity 46/46이 통과했을 뿐 아니라, 주요 트랜잭션의 **발생 타임스탬프가 동일**함을 확인했습니다 (SAN-004 첫 SG AR = 3,195,000 / SAN-006 마지막 DST AW = 5,815,000, 단위 ps). 통과 개수만으로는 "다른 타이밍으로 우연히 통과"를 배제할 수 없으므로 사이클 단위까지 비교했습니다.

---

2. 레지스터 인터페이스 공통 사양

이 장은 **모든 레지스터에 똑같이 적용되는 규칙**입니다. 버스 폭, 주소 계산법, 접근 유형 표기, 리셋 종류, 근거 등급을 정합니다.

2.1 버스와 주소 체계

항목	사양	근거
버스	AXI4-Lite slave (`s_axi_*`), 32비트 데이터	R
주소 폭	11비트 → **2 KiB (0x000–0x7FF)**	R
내부 주소	**word 주소** `up_waddr/up_raddr = s_axi_*addr[10:2]` (9비트)	R
바이트 주소 환산	`byte_offset = word_addr × 4`	R
클록 / 리셋	`s_axi_aclk` / `s_axi_aresetn` (active-low, 동기)	R
`WSTRB`	**무시.** 모든 쓰기는 32비트 전체 쓰기	R
`BRESP` / `RRESP`	**항상 `OKAY` (`2'b00`)** — 미정의 주소도 에러 없음	R
미정의 주소 읽기	**`0x0000_0000`**	**M**
미정의 주소 쓰기	무시 (부작용 없음)	R
`AxPROT`	무시	R

```
바이트 주소 = 워드 주소 x 4
  9'h000 -> 0x000      9'h100 -> 0x400      9'h11F -> 0x47C
```

본 문서의 "Offset"은 **바이트 주소**(드라이버·TB가 쓰는 값), "Word"는 RTL `case` 문의 **9비트 word 주소**입니다. 두 값이 다르므로 RTL과 대조할 때 혼동하지 않도록 양쪽을 모두 적었습니다.

⚠️ **주의 — 부분 쓰기 금지.** `WSTRB`(쓸 바이트를 고르는 신호)가 무시됩니다. 그래서 byte/halfword 쓰기를 하면 나머지 바이트가 버스에 실린 값으로 덮어써집니다. 드라이버는 **항상 32비트 단위**로 접근해야 합니다. SOR v2.0.0 **T-02** (`REG-023`)가 이 동작(1바이트 쓰기 → `0x0000_00FF`)을 확인합니다.

⚠️ **주의 — 에러 응답 부재.** 잘못된 주소 접근이 `SLVERR`(에러 응답)로 드러나지 않습니다. 검증에서 주소 디코드 오류를 잡으려면 **readback 값 비교가 유일한 수단**입니다. Sanity `SAN-007`의 미매핑 주소 체크와 SOR v2.0.0 **T-02**가 이 역할을 합니다.

2.2 접근 유형

표기	의미	쉬운 말
RO	읽기 전용. 쓰기 무시	표시등
RW	읽기/쓰기. 쓴 값이 그대로 읽힘 (필드 설명의 마스크 조건 적용)	스위치
RW1C	1을 쓰면 해당 비트의 **원천 상태**가 지워짐. 0 쓰기는 무효	"확인" 버튼
RW1S-SC	1을 쓰면 세트, 하드웨어가 조건 충족 시 자동으로 0 복귀. 0 쓰기는 무효	눌렀다 저절로 튀어나오는 버튼
RO-C	읽기 전용 상수 (파라미터로 고정)	명판
RO-RS	읽기 전용이나 **읽기에 부작용이 있음** (FIFO pop 등)	읽기만 해도 무언가 바뀜 — 주의
—	미구현. 쓰기 무시, 읽기 0	패널에 구멍만 있음

2.3 리셋 체계

레지스터는 두 종류의 "지움" 조건을 가집니다. **이 구분을 놓치면 검증 기대값이 틀립니다.**

조건	트리거	지워지는 대상
HW 리셋	`s_axi_aresetn = 0`	모든 RW 설정 레지스터 (`SCRATCH`, `IRQ_MASK`, `IRQ_SOURCE`, `CONTROL`, `FLAGS`, 주소, 길이, `SG_ADDRESS`)
채널 비활성	`CONTROL.ENABLE = 0`	전송 추적 상태 (`TRANSFER_ID`, `ACTIVE_TRANSFER_ID`, `TRANSFER_DONE`, `TRANSFER_SUBMIT`, `TRANSFER_PROGRESS`, partial FIFO) + DMA 데이터 경로 전체

쉽게 말하면, **HW 리셋은 "전원을 껐다 켜기"**, **`ENABLE=0`은 "작업만 취소하기"** 입니다. `ENABLE=0`은 **설정 레지스터를 지우지 않고**, `IRQ_SOURCE`도 지우지 않습니다. 상세는 § 8.

2.4 레지스터 상태 분류

분류	의미
Active	본 구성에서 기능함
Active (HWDESC=0)	레지스터 모드에서만 의미. SG 모드에서는 값이 무시됨
Active (HWDESC=1)	SG 모드 전용
Const	파라미터 상수를 읽기만 함
Debug	기능에 영향 없는 관측용. **판정에 쓰지 않음** (§ 4.7)
Unreachable	레지스터는 동작하나 본 구성에서 값을 바꾸는 사건이 없음
Inactive	파라미터로 비활성. 쓰기 무시, 읽기 0

2.5 근거 등급

각 값이 어디까지 확인된 것인지 구분합니다. 본 문서에서 가장 중요한 표기입니다.

등급	의미	신뢰도
M	**실측 확인** — Sanity Test(2026-10-02, 46 checks)로 readback 비교 완료	가장 높음
R	**RTL 분석** — 소스 코드에서 직접 도출. 시뮬레이션 미확인	높음
D	**upstream 문서 인용** — RTL과 어긋나는 사례가 있어 참고용	낮음

왜 이 구분이 필요한가. 통합 전 두 선행 문서는 `INTERFACE_DESCRIPTION_2`의 리셋값을 동일하게 `0x30`으로 적고 있었습니다. 양쪽 다 **RTL을 읽어서 계산**했으나, `axi_dmac_regmap.v`의 로컬 기본값(`4'b0011`)을 본 것이지 상위 모듈 `axi_dmac`이 실제로 넘기는 값을 본 것이 아니었습니다. 실측값은 `0x0000_0000`입니다. **정적 분석만으로는 이런 종류의 오류를 잡을 수 없으므로, 어떤 값이 아직 측정되지 않았는지 문서가 스스로 드러내야 합니다.**

R은 "틀렸다"가 아니라 "아직 안 봤다"입니다. 본 문서에서 R로 남은 값은 Sanity 46 checks가 건드리지 않은 것들이며, **검증팀이 측정해야 M이 됩니다** (§9.3). 특히 `X_LENGTH`는 **upstream 문서(`7`)와 RTL 분석(`0`)이 어긋난 채 미측정**이라 가장 먼저 확인할 가치가 있습니다.

v2.0.0 참고 — R 값이 처음 측정되는 곳. SOR v2.0.0의 일부 테스트는 R 값을 **처음으로 측정**합니다. 예: T-02(`*_STRIDE`, `FRAMELOCK_STRIDE` = 0), T-11/T-12(§8 매트릭스). 이 테스트가 통과해도 **이 문서의 등급은 자동으로 M이 되지 않습니다.** 결과를 확인한 뒤 다음 개정에서 올립니다. 또 이들은 "리셋 직후" 시점이 아니므로 **리셋값**의 등급은 그대로 R입니다 (§9.3).

---

3. 레지스터 맵 요약

전체 레지스터를 한 표로 봅니다. "한 줄 설명" 열은 v2.0.0에서 추가했습니다. 나머지 열은 v1.0.0과 같습니다.

Offset	Word	이름	접근	리셋값	근거	본 구성	상세	한 줄 설명
`0x000`	`0x000`	`VERSION`	RO-C	`0x0004_0565`	**M**	Const	4.1.1	IP 버전 명판
`0x004`	`0x001`	`PERIPHERAL_ID`	RO-C	`0x0000_0000`	**M**	Const	4.1.2	같은 IP 여러 개일 때 구분 번호
`0x008`	`0x002`	`SCRATCH`	RW	`0x0000_0000`	**M**	Active	4.1.3	기능 없는 연습용 칸 (버스 확인)
`0x00C`	`0x003`	`IDENTIFICATION`	RO-C	`0x444D_4143`	**M**	Const	4.1.4	"DMAC" 글자 명판
`0x010`	`0x004`	`INTERFACE_DESCRIPTION_1`	RO-C	`0x3807_0303`	**M**	Const	4.1.5	빌드 설정 요약 1
`0x014`	`0x005`	`INTERFACE_DESCRIPTION_2`	RO-C	**`0x0000_0000`**	**M**	Const	4.1.6	빌드 설정 요약 2 (버스 캐시·보호 속성)
`0x080`	`0x020`	`IRQ_MASK`	RW	**`0x0000_0003`**	**M**	Active	4.2.1	알림 막기 스위치 (1 = 막음)
`0x084`	`0x021`	`IRQ_PENDING`	RW1C	`0x0000_0000`	**M**	Active	4.2.2	지금 올라온 알림. 1 쓰면 지움
`0x088`	`0x022`	`IRQ_SOURCE`	RO	`0x0000_0000`	**M**	Active	4.2.3	막힘과 무관하게 일어난 일 기록
`0x400`	`0x100`	`CONTROL`	RW	`0x0000_0000`	**M**	Active	4.3.1	켜기 / 일시정지 / SG 모드
`0x404`	`0x101`	`TRANSFER_ID`	RO	`0x0000_0000`	**M**	Active	4.4.1	다음 출발에 붙을 번호 (0~3)
`0x408`	`0x102`	`TRANSFER_SUBMIT`	RW1S-SC	`0x0000_0000`	R	Active	4.4.2	출발 버튼 (저절로 0 복귀)
`0x40C`	`0x103`	`FLAGS`	RW	**`0x0000_0002`**	**M**	Active (일부)	4.4.3	전송 옵션 (TLAST 기본 1)
`0x410`	`0x104`	`DEST_ADDRESS`	RW	`0x0000_0000`	**M**	Active (HWDESC=0)	4.4.4	목적지 주소 (레지스터 모드용)
`0x414`	`0x105`	`SRC_ADDRESS`	RW	`0x0000_0000`	**M**	Active (HWDESC=0)	4.4.5	소스 주소 (레지스터 모드용)

Offset	Word	이름	접근	리셋값	근거	본 구성	상세	한 줄 설명
`0x418`	`0x106`	`X_LENGTH`	RW	`0x0000_0000`	**R** [3]	Active (HWDESC=0)	4.4.6	길이 = 바이트 수 − 1 (레지스터 모드용)
`0x41C`	`0x107`	`Y_LENGTH`	—	`0x0000_0000`	**M**	Inactive	4.8	2D용, 꺼짐
`0x420`	`0x108`	`DEST_STRIDE`	—	`0x0000_0000`	R	Inactive	4.8	2D용, 꺼짐
`0x424`	`0x109`	`SRC_STRIDE`	—	`0x0000_0000`	R	Inactive	4.8	2D용, 꺼짐
`0x428`	`0x10A`	`TRANSFER_DONE`	RO	`0x0000_0000`	**M**	Active	4.5.1	번호별 완료 표시등
`0x42C`	`0x10B`	`ACTIVE_TRANSFER_ID`	RO	`0x0000_0000`	**R** [3]	Active	4.5.2	다음에 끝날 전송의 번호
`0x430`	`0x10C`	`STATUS`	RO	`0x0000_0000`	R	예약 (항상 0)	4.5.3	쓰이지 않음
`0x434`	`0x10D`	`CURRENT_DEST_ADDRESS`	RO	가변 [1]	R	Debug	4.7.1	(관찰용) 지금 쓰는 주소
`0x438`	`0x10E`	`CURRENT_SRC_ADDRESS`	RO	가변 [1]	R	Debug	4.7.2	(관찰용) 지금 읽는 주소
`0x43C`	`0x10F`	`DBG_STATUS`	RO	가변 [2]	R	Debug	4.7.3	(관찰용) 내부 리셋 상태 머신
`0x440`	`0x110`	`DBG_IDS0`	RO	`0x0000_0000`	R	Debug (Gray)	4.7.4	(관찰용) 목적지 쪽 burst 순번
`0x444`	`0x111`	`DBG_IDS1`	RO	`0x0000_0000`	R	Debug (Gray)	4.7.4	(관찰용) 소스 쪽 burst 순번
`0x448`	`0x112`	`TRANSFER_PROGRESS`	RO	`0x0000_0000`	R	Active	4.5.4	지금까지 옮긴 바이트 수
`0x44C`	`0x113`	`PARTIAL_TRANSFER_LENGTH`	RO	`0x0000_0000`	R	**Unreachable**	4.5.5	이 구성에서 쓸 일 없음
`0x450`	`0x114`	`PARTIAL_TRANSFER_ID`	**RO-RS**	`0x0000_0000`	R	**Unreachable**	4.5.5	⚠️ **읽기 금지** (읽으면 FIFO pop)
`0x454`	`0x115`	**`DESCRIPTOR_ID`**	RO	`0x0000_0000`	**M**	**Active (HWDESC=1)**	4.6.2	처리 중인 지시서의 `id` (디버깅용)
`0x458`	`0x116`	`FRAMELOCK_CONFIG`	—	`0x0000_0000`	**M**	Inactive	4.8	프레임락용, 꺼짐
`0x45C`	`0x117`	`FRAMELOCK_STRIDE`	—	`0x0000_0000`	R	Inactive	4.8	프레임락용, 꺼짐
`0x47C`	`0x11F`	**`SG_ADDRESS`**	RW	`0x0000_0000`	**M**	**Active (HWDESC=1)**	4.6.1	첫 지시서가 있는 메모리 주소
`0x490`	`0x124`	`DEST_ADDRESS_HIGH`	—	`0x0000_0000`	**M**	**Inactive**	§ 10	64비트 주소용, 꺼짐
`0x494`	`0x125`	`SRC_ADDRESS_HIGH`	—	`0x0000_0000`	**M**	**Inactive**	§ 10	64비트 주소용, 꺼짐
`0x498`	`0x126`	`CURRENT_DEST_ADDRESS_HIGH`	RO	`0x0000_0000`	R	**Inactive**	§ 10	64비트 주소용, 꺼짐
`0x49C`	`0x127`	`CURRENT_SRC_ADDRESS_HIGH`	RO	`0x0000_0000`	R	**Inactive**	§ 10	64비트 주소용, 꺼짐
`0x4BC`	`0x12F`	`SG_ADDRESS_HIGH`	—	`0x0000_0000`	**M**	**Inactive**	§ 10	64비트 주소용, 꺼짐
그 외	—	(미정의)	—	`0x0000_0000`	**M**	—	§ 2.1	읽으면 0, 에러 없음

[1] AXI master 포트의 주소 출력을 그대로 읽으므로 리셋 직후 값은 address generator 상태에 따릅니다. **리셋값 비교 대상에서 제외**를 권장합니다. [2] reset manager 상태 머신 값이 그대로 보이므로 고정 리셋값이 없습니다. 4.7.3 참조. [3] **미측정.** Sanity 범위 밖이고 **SOR 요구사항도 없습니다**(SOR v2.0.0에도 **리셋값**을 재는 테스트는 없음) → § 9.3. `ACTIVE_TRANSFER_ID`는 SOR v2.0.0 T-10·T-11이 **전송 중·끈 뒤 값**을 판정하지만, 리셋 직후 값은 아닙니다.

SG 고유 레지스터 2개: `SG_ADDRESS`(0x47C), `DESCRIPTOR_ID`(0x454). 두 레지스터는 `DMA_SG_TRANSFER = 1`일 때만 `generate` 블록으로 생성됩니다.

> **`*_HIGH` 레지스터군이 Inactive인 이유.** `DMA_AXI_ADDR_WIDTH = 32`(결정 2)에서는 `if (HAS_ADDR_HIGH)` 가드로 **쓰기 로직 자체가 합성되지 않습니다.** 레지스터가 "0을 반환"하는 것이 아니라 **존재하지 않습니다.** Sanity `SAN-007`이 `0xFFFF_FFFF` 기입

> 후 `0x0000_0000` 읽힘을 실측했습니다.

---

4. 레지스터 상세

각 레지스터는 **한 줄 요약 → 비트 필드 → 동작 → RTL 근거 → 본 구성 참고 → 주의·검증 포인트** 순서입니다. "SOR" 표시는 SOR v2.0.0 테스트 번호(T-xx)이며, 괄호 안은 원래 요구사항 ID입니다.

4.1 식별 · 구성 정보 (0x000–0x014)

이 묶음은 **명판**입니다. 값이 고정되어 있고, "버스가 살아 있는가"와 "의도한 설정으로 만들어졌는가"를 확인하는 데 씁니다.

4.1.1 `VERSION` (0x000)

한 줄 요약: IP 버전 번호. 가장 먼저 읽는 레지스터입니다.

Bits	Field	접근	값	설명
`[31:16]`	`VERSION_MAJOR`	RO-C	`0x0004`	주 버전 4
`[15:8]`	`VERSION_MINOR`	RO-C	`0x05`	부 버전 5
`[7:0]`	`VERSION_PATCH`	RO-C	`0x65`	패치

- **RTL 근거**: `axi_dmac_regmap.v:150 localparam PCORE_VERSION = 'h00040565`, `:248`.
- **실측 (M)**: `0x0004_0565` — `SAN-001` check 1.
- **검증 포인트**: Sanity의 첫 readback입니다. **이 값이 틀리면 버스나 레지스터 디코딩이 깨진 것**이므로 이후 테스트를 진행할 의미가 없습니다. TB가 이 체크 실패 시 나머지를 SKIP 처리합니다. SOR **T-01** (`REG-001`)도 같은 규칙(틀리면 나머지 테스트를 모두 건너뜀)을 씁니다.

4.1.2 `PERIPHERAL_ID` (0x004)

한 줄 요약: 같은 DMA를 여러 개 붙였을 때 서로 구분하는 번호.

Bits	Field	접근	값	설명
`[31:0]`	`PERIPHERAL_ID`	RO-C	`ID` 파라미터	동일 IP 다중 인스턴스 식별용

- **RTL 근거**: `axi_dmac_regmap.v:249`.
- **실측 (M)**: `0x0000_0000` — `sgdma_top`이 `ID = 0`으로 명시.
- **SoC 통합 시**: 여러 SGDMA를 붙이면 인스턴스별로 다른 `ID`를 주고, 이 레지스터로 구분합니다.
- ⚠️ **검증자 주의**: 값이 틀리면 버스 고장보다 **기대값이 틀렸을 가능성**(파라미터 변경)을 먼저 봅니다 (SOR T-01).

4.1.3 `SCRATCH` (0x008)

한 줄 요약: 아무 기능 없는 연습용 칸. 써 보고 읽어서 버스 쓰기 경로를 확인합니다.

Bits	Field	접근	리셋	설명
`[31:0]`	`SCRATCH`	RW	`0x0000_0000`	기능 없음. 버스 연결 확인용

- **RTL 근거**: `axi_dmac_regmap.v:169`(선언), `:213`(리셋), `:220-222`(쓰기), `:250`(읽기).
- **실측 (M)**: `0xA5A5_5A5A` / `0x5A5A_A5A5` / `0x0000_0000` 3회 write-readback 일치 — `SAN-002`.
- **검증 포인트**: **패턴을 반전해 두 번 쓰는 것**이 중요합니다. 한 패턴만 쓰면 일부 비트가 고착(stuck-at, 고장으로 0이나 1에 붙어 있음)되어 있어도 통과합니다. `WSTRB`가 무시되므로 byte-strobe 테스트는 "무시됨"을 기대값으로 둡니다 (SOR **T-02** (`REG-023`)).

4.1.4 `IDENTIFICATION` (0x00C)

한 줄 요약: ASCII 글자 "DMAC"이 새겨진 명판.

Bits	Field	접근	값	설명
`[31:0]`	`IDENTIFICATION`	RO-C	`0x444D_4143`	ASCII `"DMAC"`

- **RTL 근거**: `axi_dmac_regmap.v:251`.
- **실측 (M)**: `0x444D_4143` — `SAN-001` check 2. `VERSION`과 함께 버스 생존 판정 기준입니다.

4.1.5 `INTERFACE_DESCRIPTION_1` (0x010)

한 줄 요약: 빌드 설정(파라미터)을 한 단어에 묶어 보여 주는 상수. 한 번 읽으면 "의도한 구성으로 만들어졌는가"를 알 수 있습니다.

합성 파라미터를 소프트웨어가 확인할 수 있도록 패킹한 상수 레지스터입니다.

```
 31    27 26 25 24 23   20 19   16 15 14 13 12 11    8 7  6 5  4 3    0
+--------+--+--+--+------+------+-----+-----+------+----+-----+------+
|MAX_NUM_|2D|EX|AR| rsvd |BPBRST|rsvd |TYPE | BPB  |rsvd|TYPE | BPB  |
|FRAMES-1|TL|SY|UN|      |_WIDTH|     | SRC | SRC  |    |DEST | DEST |
+--------+--+--+--+------+------+-----+-----+------+----+-----+------+
    7     0  0  0    0      7      0     0     3     0     0     3    -> 0x3807_0303
```

Bits	Field	접근	본 구성 값	설명
`[31:27]`	`MAX_NUM_FRAMES`	RO-C	`7`	`2**MAX_NUM_FRAMES_WIDTH - 1`. FRAMELOCK=0이어도 파라미터 값이 그대로 보임
`[26]`	`DMA_2D_TLAST_MODE`	RO-C	`0`	2D TLAST 모드
`[25]`	`USE_EXT_SYNC`	RO-C	`0`	외부 sync 사용 여부
`[24]`	`HAS_AUTORUN`	RO-C	`0`	AUTORUN 파라미터
`[23:20]`	—	RO	`0`	예약
`[19:16]`	`BYTES_PER_BURST_WIDTH`	RO-C	`7`	`log2(128)`
`[15:14]`	—	RO	`0`	예약
`[13:12]`	`DMA_TYPE_SRC`	RO-C	`0`	0 = AXI-MM, 1 = AXI-Stream, 2 = FIFO
`[11:8]`	`BYTES_PER_BEAT_SRC_LOG2`	RO-C	`3`	`log2(64/8)`
`[7:6]`	—	RO	`0`	예약
`[5:4]`	`DMA_TYPE_DEST`	RO-C	`0`	0 = AXI-MM
`[3:0]`	`BYTES_PER_BEAT_DEST_LOG2`	RO-C	`3`	`log2(64/8)`

- **RTL 근거**: `axi_dmac_regmap.v:252-255`.
- **실측 (M)**: **`0x3807_0303`** — 계산값과 일치. `SAN-001` check 4.
- **본 구성 참고**: **레지스터 한 번 읽기로 빌드 파라미터 전체를 확인**할 수 있는 지점입니다. `DMA_TYPE_SRC/DEST`, `BYTES_PER_BEAT`, `MAX_BYTES_PER_BURST`, `MAX_NUM_FRAMES`가 모두 인코딩되어 있어, 이 값이 맞으면 **"의도한 구성으로 합성됐다"가 증명**됩니다. SoC 통합 후 "IP가 잘못 빌드된 것 아닌가" 의심이 들 때 가장 먼저 읽을 레지스터입니다.

- **RTL 해석 주의**: `[31:27]` 식은 `MAX_NUM_FRAMES[4:0]-1`로, 크기 없는 상수 `1` 때문에 self-determined 폭이 32비트가 됩니다. 결과적으로 `[31:27]`에는 하위 5비트(=7)가 정상적으로 남지만 **린트 경고가 날 수 있습니다.**

4.1.6 `INTERFACE_DESCRIPTION_2` (0x014)

한 줄 요약: DMA가 버스 요청에 붙이는 캐시·보호 속성(`AxCACHE`/`AxPROT`) 설정값을 보여 주는 상수. 본 구성에서는 `0x0000_0000`.

> ⚠️ **결정 1·4가 코드에 적용된 레지스터입니다 (2026-10-06).** 그 전까지 두 파라미터는 래퍼에 미지정이었고, 그 때문에 선행 문서 2건이 모두 리셋값을 `0x30`으로 잘못 적었습니다. **`sgdma_top`이 `AXI_AXCACHE = 4'b0000`을 명시하므로, 이제 이 값은 코드에 의도로 남아 있습니다.** 명시 전후로 레지스터 값과 AXI 타임스탬프가 완전히 동일함을 확인했습니다.

Bits	Field	접근	확정값	설명
`[31:11]`	—	RO	`0`	예약
`[10:8]`	`AXI_AXPROT`	RO-C	**`3'b000`**	모든 master 포트의 `AxPROT` 출력값
`[7:4]`	`AXI_AXCACHE`	RO-C	**`4'b0000`**	모든 master 포트의 `AxCACHE` 출력값
`[3:1]`	—	RO	`0`	예약
`[0]`	`CACHE_COHERENT`	RO-C	`0`	캐시 일관성 모드

- **RTL 근거**: `axi_dmac_regmap.v:256-259`.
- **실측 (M)**: **`0x0000_0000`** — `SAN-001` check 5 (2026-10-01).
- **과거 오류의 원인**: `axi_dmac_regmap.v`는 `parameter [3:0] AXI_AXCACHE = 4'b0011`을 **자체 기본값**으로 갖습니다. 두 선행 문서 모두 이 선언을 보고 `0x30`을 계산했으나, 실제로는 상위 모듈 `axi_dmac`이 넘기는 값이 적용됩니다. **모듈 선언의 기본값은 그 모듈이 top-level일 때만 쓰인다**는 점을 놓친 사례입니다.

`AXCACHE = 4'b0000`의 의미와 범위 (결정 1)

항목	내용
AXI 의미	`AxCACHE[3:0] = 0000` = **Device Non-bufferable**. slave가 write를 버퍼링하거나 재배열하지 않아야 함
IP DV 관점	**가장 결정론적**입니다. 메모리 모델의 응답 순서가 고정되므로 Sanity의 사이클 단위 비교가 성립합니다. 기존 46개 체크의 기대값이 **하나도 바뀌지 않습니다.**
SoC 통합 관점	MM→MM DMA 목적지가 Normal memory라면 통상 `4'b0011`(Normal Non-cacheable Bufferable)을 씁니다. `0000`은 interconnect가 write를 직렬화할 수 있어 **대역폭이 떨어질 수 있습니다.**
검증	**명시 전후 Sanity 46/46 + AXI 타임스탬프 19개 전부 동일.** 레지스터 값도 `0x0000_0000` 불변 — 결정이 동작을 바꾸지 않았음이 실측으로 확인되었습니다
결론	**IP DV 단계는 `4'b0000`으로 확정.** 성능 관점의 재검토는 **SoC DV 이관 항목**으로 남깁니다 → § 11 #1

- ⚠️ **검증 포인트 — 레지스터 값 ≠ 버스 출력**: 이 레지스터는 **파라미터 값을 그대로 복사**하는 것이므로, 레지스터가 맞다고 버스 출력이 맞는 것은 아닙니다. `m_src_axi_arcache` / `m_dest_axi_awcache` / `m_sg_axi_arcache`가 실제로 `4'b0000`을 내는지는 **AXI 모니터로 별도 확인**해야 합니다. SOR **T-14** (`AXI-007`)가 세 포트의 `AxCACHE`/`AxPROT`를 assertion으로 검사합니다 (TB 보강 B1 필요).

4.2 인터럽트 (0x080–0x088)

세 레지스터가 하나의 구조를 이룹니다. 비유하면 **`IRQ_SOURCE`는 일지, `IRQ_MASK`는 "이 종류는 벨을 울리지 마" 스위치, `IRQ_PENDING`은 "지금 벨이 울리는 이유" 표시등**입니다.

```
이벤트 -> IRQ_SOURCE(래치) -> IRQ_MASK로 게이트 -> IRQ_PENDING -> irq 출력
```

Bit	이벤트	내부 신호	발생 조건
`[0]`	`TRANSFER_QUEUED`	`up_sot`	전송 요청이 DMA 코어에 수락됨 ("접수")

Bit	이벤트	내부 신호	발생 조건
[1]	TRANSFER_COMPLETED	up_eot	전송(또는 EOT 플래그 descriptor)이 완료됨 ("완료")

> ⚠️ **비트0 "접수"는 완료가 아닙니다.** 출발 요청이 받아들여졌다는 뜻일 뿐, 데이터는 아직 옮겨지지 않았습니다. 완료는 비트1입니다 → § 6.4.

4.2.1 IRQ_MASK (0x080)

한 줄 요약: 알림 막기 스위치. 비트가 **1이면 막힘**. 리셋 직후에는 둘 다 막혀 있습니다(0x3).

Bits	Field	접근	리셋	설명
[31:2]	—	RO	0	예약
[1]	TRANSFER_COMPLETED	RW	1	1 = 마스크(차단), 0 = 허용
[0]	TRANSFER_QUEUED	RW	1	1 = 마스크, 0 = 허용

- **RTL 근거**: axi_dmac_regmap.v:176 reg [1:0] up_irq_mask = 2'h3, :212(리셋), :223-225(쓰기), :260(읽기).
- **실측 (M)**: 리셋값 0x0000_0003 — SAN-003 check 2.
- **본 구성 참고**: **리셋 시 두 인터럽트 모두 마스크**되어 있습니다. 드라이버가 명시적으로 unmask(막기 해제)해야 irq가 올라옵니다. **FLAGS와 함께 비영 리셋값을 갖는 두 레지스터 중 하나입니다.**
- ⚠️ **드라이버/검증자 주의**
  - 마스크는 IRQ_SOURCE 래치를 **막지 않습니다.** 마스크 상태에서도 이벤트는 쌓이고, **unmask하는 순간 irq가 즉시 올라갑니다** (과거 이벤트가 튀어나옴). unmask 전에 IRQ_PENDING에 0x3을 써서 비우는 순서를 드라이버·테스트 모두 지켜야 합니다.
  - **IRQ_MASK = 0x0(둘 다 허용)이면 출발 직후 "접수(TRANSFER_QUEUED)" 알림이 먼저 올라옵니다.** 첫 irq를 완료로 오인하지 마십시오. 표준 설정은 0x1(접수는 막고 완료만 허용)입니다 → § 6.4.
  - 마스크 시험에서 irq를 기다리면 막힌 설정(0x3, 0x2)에서는 완료 알림이 오지 않아 시간 초과가 납니다. 완료는 **TRANSFER_DONE을 반복해서 읽어** 기다립니다 (SOR **T-09**).
  - unmask 케이스를 최소 1회 포함해야 IRQ 경로가 검증됩니다. Sanity SAN-006이 IRQ_MASK = 0x1로 EOT만 unmask하여 이를 수행합니다. 네 가지 조합 전체는 SOR **T-09** (IRQ-002 ~ IRQ-005)가 봅니다.

4.2.2 IRQ_PENDING (0x084)

한 줄 요약: 지금 irq를 올리고 있는 알림. 1을 쓰면 지워지는데, **실제로 지워지는 곳은 IRQ_SOURCE** 입니다.

Bits	Field	접근	리셋	설명
[31:2]	—	RO	0	예약
[1]	TRANSFER_COMPLETED	RW1C	0	읽기: SOURCE[1] & ~MASK[1]. 1 쓰기: **IRQ_SOURCE[1]** 지움
[0]	TRANSFER_QUEUED	RW1C	0	읽기: SOURCE[0] & ~MASK[0]. 1 쓰기: **IRQ_SOURCE[0]** 지움

```
assign up_irq_pending      = ~up_irq_mask & up_irq_source;                          // :184
assign up_irq_source_clear = (up_wreq && up_waddr == 9'h021) ? up_wdata[1:0] : 2'b00;  // :186
up_irq_source <= up_irq_trigger | (up_irq_source & ~up_irq_source_clear);            // :200
```

- **실측 (M)**: 리셋값 0x0(SAN-003), 전송 후 [1] 세트 → W1C → 0x0 복귀(SAN-006).
- **동작 상세**
  - IRQ_PENDING은 **독립 저장소가 아닙니다.** IRQ_SOURCE와 IRQ_MASK의 조합 결과입니다.
  - 1 쓰기는 **마스크 여부와 무관하게** IRQ_SOURCE를 지웁니다. 마스크된 이벤트도 이 레지스터로 지울 수 있습니다.
  - 같은 사이클에 새 이벤트와 clear가 겹치면 **이벤트가 이깁니다** — 이벤트를 잃지 않는 설계입니다.

> ⚠️ **이름과 실제가 어긋나는 지점입니다.** "PENDING에 쓴다"고 생각하지만 **지워지는 것은 SOURCE**입니다. 검증 시나리오를 쓸 때 반드시 구분해야 합니다 (SOR **T-09** 3단계).

4.2.3 `IRQ_SOURCE` (0x088)

한 줄 요약: 막혀 있든 아니든 일어난 일을 모두 적는 일지. HW 리셋이나 `IRQ_PENDING` 1 쓰기로만 지워집니다.

Bits	Field	접근	리셋	설명
`[31:2]`	—	RO	`0`	예약
`[1]`	`TRANSFER_COMPLETED`	RO	`0`	`up_eot` 발생 시 래치. `IRQ_PENDING[1]`에 1을 써야 지워짐
`[0]`	`TRANSFER_QUEUED`	RO	`0`	`up_sot` 발생 시 래치. `IRQ_PENDING[0]`에 1을 써야 지워짐

- **RTL 근거**: `axi_dmac_regmap.v:196-202`, `:262`.
- **실측 (M)**: 리셋값 `0x0`(`SAN-003`). **전송 후 W1C로 `[1]`만 지우면 `[0]`은 `0x1`로 남음** — `SAN-006` check 4. 마스크된 SOT 이벤트가 SOURCE에는 기록된다는 직접 증거입니다.
- **본 구성 참고**: `s_axi_aresetn`으로만 지워집니다. **`CONTROL.ENABLE=0`으로는 지워지지 않습니다.**
- **검증 포인트**: 마스크와 무관하게 기록되므로, 인터럽트를 쓰지 않는 polling 테스트에서도 이 레지스터로 SOT/EOT 발생을 확인할 수 있습니다. SOR v2.0.0의 공통 조건 C4는 "알림을 **지우기 전에** 읽은 `IRQ_PENDING[1]` = 1"이고, 알림을 막아 둔 시험에서는 이 레지스터의 `IRQ_SOURCE[1]`(또는 `TRANSFER_DONE` 비트)로 봅니다. `IRQ_PENDING`에 쓰면 여기도 지워지므로, 지운 뒤에 보면 안 됩니다.
- ⚠️ **검증자 주의**: 읽기 전용입니다. 직접 써도 바뀌지 않고 응답은 OKAY입니다 (SOR **T-02** (`REG-019`)).

4.3 채널 제어 (0x400)

4.3.1 `CONTROL` (0x400)

한 줄 요약: DMA의 주 스위치. 비트0 켜기(ENABLE), 비트1 일시정지(PAUSE), 비트2 SG 모드(HWDESC). SG를 쓰려면 `0x5`를 씁니다.

Bits	Field	접근	리셋	본 구성 동작
`[31:4]`	—	RO	`0`	예약
`[3]`	`FRAMELOCK`	**RO**	`0`	쓰기값 `&` `FRAMELOCK(=0)` → **항상 0**
`[2]`	`HWDESC`	RW	`0`	**1 = Scatter-Gather 모드.** 쓰기값 `&` `DMA_SG_TRANSFER(=1)`
`[1]`	`PAUSE`	RW	`0`	1 = 일시정지(상태 보존)
`[0]`	`ENABLE`	RW	`0`	1 = 채널 활성, 0 = 비활성 + 내부 상태 초기화

```
        ctrl_flock  <= up_wdata[3] & FRAMELOCK;        // = 0
        ctrl_hwdesc <= up_wdata[2] & DMA_SG_TRANSFER;  // 반영됨
        ctrl_pause  <= up_wdata[1];
        ctrl_enable <= up_wdata[0];                    // :226-231
```

- **실측 (M)**: 리셋값 `0x0`(`SAN-003`). **`0xFFFF_FFFF` 기입 → `0x0000_0007` 읽힘**(`SAN-007`) — `[3]` 마스킹 확인.
- **동작 상세**
  - **`ENABLE`** 1→0 전이(단 1사이클이라도)는 reset manager의 `needs_reset`을 세웁니다. 진행 중인 AXI 트랜잭션을 프로토콜상 완료한 뒤 모든 도메인에 리셋 펄스를 넣습니다. 동시에 regmap의 전송 추적 상태가 즉시 지워집니다 (§ 8).
  - **`PAUSE`** = 1이면 `ENABLED → SHUTDOWN → DISABLED`로 가지만 **리셋 펄스는 내지 않습니다.** 0으로 되돌리면 `STARTUP`을 거쳐 이전 상태에서 재개합니다.
  - **`HWDESC`** 는 전송 요청의 출처를 고릅니다. 0이면 레지스터, 1이면 `SG_ADDRESS`가 가리키는 descriptor 체인입니다.
- **본 구성 참고 — HWDESC 리셋값은 0입니다.** "SG 모드 상시 사용"은 **소프트웨어 운용 전제**이지 하드웨어 리셋값이 아닙니다. SG 모드를 쓰려면 드라이버가 `CONTROL`에 **`0x5` (ENABLE | HWDESC)** 를 써야 합니다.

- **자주 쓰는 값** (SOR v2.0.0에서 쓰는 값)

쓰는 값	뜻	쓰는 곳
`0x0`	끄기 (모든 비트 0)	테스트 시작 전 공통 준비
`0x5`	켜기 + SG 모드	표준 순서
`0x4`	**켜기만 끄고** SG 모드 유지	"hang 없음" 절차, T-11
`0x7`	켜기 + 일시정지 + SG 모드	T-12

- ⚠️ **프로그래밍 규칙**
  - 네 비트가 **한 번의 쓰기로 동시에** 갱신됩니다. `PAUSE`만 바꿀 때도 `HWDESC`/`ENABLE`을 함께 실어야 합니다(read-modify-write, 읽고 고쳐서 다시 쓰기).
  - `HWDESC`는 **`ENABLE`과 같은 쓰기로** 설정하고, 전송 중에는 바꾸지 않습니다. 전송 도중 바꾸면 요청·응답 경로 mux가 즉시 전환되어 **진행 중인 전송의 응답 매칭이 깨질 수 있습니다.** RTL에 이를 막는 보호 로직은 없습니다.
  - `0xFFFF_FFFF`를 쓰면 켜기·일시정지·SG 모드가 모두 1이 됩니다. 이 상태에서 `TRANSFER_SUBMIT`을 쓰지 말고 먼저 되돌립니다 (SOR **T-02**).
- **검증 포인트**
  - `FRAMELOCK` 비트에 1을 써도 0으로 읽히는지 (`SAN-007` — **실측 완료**, SOR **T-02** (`REG-021`)).
  - `ENABLE` 0→1→0 토글 후 §8의 지워짐 대상이 모두 0인지 (**미측정** — SOR **T-11** (`REG-006`, v1.0.0 시나리오 `FT-011`)).
  - `PAUSE` 후 상태 보존·재개 (**미측정** — SOR **T-12** (`REG-008`, v1.0.0 시나리오 `FT-012`)).

4.4 전송 요청 (0x404–0x424)

4.4.1 `TRANSFER_ID` (0x404)

한 줄 요약: 다음에 출발할 전송에 붙을 번호(0~3). SG 모드에서는 **체인 하나 = 번호 하나**입니다.

Bits	Field	접근	리셋	설명
`[31:2]`	—	RO	`0`	예약
`[1:0]`	`TRANSFER_ID`	RO	`0`	**다음에 제출될** 전송의 ID (2비트, 0–3 순환)

- **RTL 근거**: `axi_dmac_regmap_request.v:133`, `:364-373`, `:210`.
- **실측 (M)**: 리셋값 `0x0` — `SAN-003` check 7.
- **동작 상세**: `up_sot`마다 +1 (binary, mod 4). 제출 시점에 `TRANSFER_DONE`의 해당 비트를 0으로 내립니다. `ENABLE=0`이면 0으로 초기화.
- **본 구성 참고**: 이 ID는 **소프트웨어용 2비트 binary 카운터**이며, 내부 Gray-ID(4비트)와는 **무관**합니다. SG 모드에서는 **체인 1개 제출 = ID 1개**입니다.
- **SOR 연결**: 체인 하나에 번호가 1만 늘어나는지는 **T-04** (`FUNC-002`), 연속 예약 때 B의 번호가 `ID_A + 1`이고 두 체인 뒤 `TRANSFER_ID = ID_A + 2`인지는 **T-10** (`REG-012`), `ENABLE=0` 후 0은 **T-11** (`REG-006`).

4.4.2 `TRANSFER_SUBMIT` (0x408)

한 줄 요약: 출발 버튼. 1을 쓰면 출발 요청, 받아들여지면 **저절로 0**. 켜기(`ENABLE`) 전에 누르면 **조용히 무시**됩니다.

Bits	Field	접근	리셋	설명
`[31:1]`	—	RO	`0`	예약
`[0]`	`TRANSFER_SUBMIT`	**RW1S-SC**	`0`	1 쓰기 = 전송 요청 발행. 코어가 수락하면(`up_sot`) 0으로 복귀

```
        if (ctrl_enable == 1'b1) begin
          if (up_wreq && up_waddr == 9'h102) up_dma_req_valid <= up_dma_req_valid | up_wdata[0];
          else if (up_sot) up_dma_req_valid <= 1'b0;
        end else begin
          up_dma_req_valid <= 1'b0;       // <- ENABLE=0이면 SUBMIT 무시
        end                                // :175-183
```

- **동작 상세**
  - **`ENABLE=1`일 때만 쓰기가 반영됩니다.** `ENABLE=0`에서의 쓰기는 무시되고 비트도 0으로 유지됩니다. **조용히 실패**하므로 순서가 중요합니다.
  - 0 쓰기는 효과가 없습니다(`| up_wdata[0]`). **이미 1인 상태에서 다시 1을 써도 요청이 하나 더 쌓이지 않습니다.** 두 번째 요청은 사라집니다.
  - 읽기값 1 = "요청이 아직 코어에 수락되지 않음".
- ⚠️ **프로그래밍 규칙**: 쓰기 전에 반드시 이 비트가 **0이 될 때까지 polling**(반복해서 읽기) 합니다.
- **SG 모드에서의 수락 시점**: HWDESC=1이면 수락 신호는 SG 주소 FIFO(`dmac_sg.v`, 1단)의 ready입니다. **첫 descriptor 주소가 SG 엔진에 넘어간 시점**에 0으로 돌아오며, 체인 전체가 끝날 때까지 기다리지 않습니다.
- ⚠️ **검증자 주의 — "1인 동안 다시 쓰기" 시험**: 수락이 빨라서 1인 시간 창이 보장되지 않습니다. 쓰기 직후 다시 읽어 **1일 때만** 중복 요청 시험을 판정하고, 0이면 그 회차를 "무효"로 기록합니다 (SOR **T-10** 3단계).
- **미측정 (R)**: 현재 Sanity는 `ENABLE=1` 상태에서 1회 쓰기만 합니다. self-clearing 타이밍과 `ENABLE=0` 시 무시 동작은 SOR **T-10** (`REG-009` ~ `REG-011`, v1.0.0 시나리오 `FT-007`/`FT-014`)이 처음 측정합니다.

4.4.3 `FLAGS` (0x40C)

한 줄 요약: 전송 옵션. 리셋값이 **`0x2`**(TLAST = 1)로, 0이 아닙니다.

> ⚠️ **리셋값이 `0`이 아닙니다.** "리셋 후 전 레지스터 0"을 가정한 테스트는 여기서 FAIL합니다.

Bits	Field	접근	리셋	본 구성 동작
`[31:3]`	—	RO	`0`	예약
`[2]`	`PARTIAL_REPORTING_EN`	RW	`0`	1 = partial 전송 길이 기록 활성
`[1]`	`TLAST`	RW	**`1`**	전송 마지막 표시(`request_last`)
`[0]`	`CYCLIC`	**RO**	`0`	쓰기값 `&` `CYCLIC(=0)` → **항상 0**

```
        localparam AUTORUN_FLAGS_LAST = AUTORUN ? AUTORUN_FLAGS[1] : 1;   // <- 1
        up_dma_cyclic <= up_wdata[0] & DMA_CYCLIC;                        // <- 상시 0  :188
```

- **실측 (M)**: 리셋값 **`0x0000_0002`**(`SAN-003` check 5). **`0xFFFF_FFFF` 기입 → `0x0000_0006` 읽힘**(`SAN-007`) — `[0]` 마스킹 확인.
- **본 구성 참고**
  - **`CYCLIC`**: 파라미터가 쓰기 경로를 마스크하므로 **레지스터 수준에서 이미 0 고정**입니다. 레지스터 비트 자체를 RTL에서 제거할 실익이 크지 않다고 판단한 근거입니다.
  - **`TLAST`**: HWDESC 모드에서도 descriptor가 아니라 **이 레지스터 값**이 요청에 실립니다. 목적지가 AXI-MM이라 버스에 TLAST가 없으므로 데이터 경로 영향은 없으나, 내부 완료 처리에 쓰이므로 **리셋값 1을 유지**하는 것을 권장합니다.
  - **`PARTIAL_REPORTING_EN`**: partial 전송은 AXI-Stream 소스의 조기 TLAST로만 발생합니다. 소스가 AXI-MM인 본 구성에서는 **1로 설정해도 기록될 사건이 없습니다.**
- ⚠️ **검증자 주의**: `0xFFFF_FFFF`를 써 본 뒤에는 반드시 `0x2`로 되돌립니다 (`SAN-007`, SOR **T-02** 5단계).

4.4.4 `DEST_ADDRESS` (0x410)

한 줄 요약: 레지스터 모드(HWDESC=0)의 목적지 주소. 아래 3비트는 버려집니다(8바이트 단위). SG 모드에서는 쓰이지 않습니다.

Bits	Field	접근	리셋	설명
`[31:3]`	`DEST_ADDRESS[31:3]`	RW	`0`	목적지 시작 주소

Bits	Field	접근	리셋	설명
`[2:0]`	—	**RO**	`0`	beat 정렬 비트. 쓰기 무시, 읽기 0

- **RTL 근거**: `axi_dmac_regmap_request.v:192`(쓰기), `:213`(읽기 — `{addr[31:3], 3'b000}`).
- **실측 (M)**: 리셋값 `0x0`(`SAN-003`). **`0xFFFF_FFFF` 기입 → `0xFFFF_FFF8` 읽힘**(`SAN-007`).
- **정렬**: 64비트 버스이므로 **8바이트 정렬**이 필요합니다. 비정렬 값을 쓰면 하위 3비트가 **조용히 버려집니다.** upstream은 4 KiB 경계 넘김을 피하기 위해 `MAX_BYTES_PER_BURST`(128 B) 정렬을 권장합니다.
- **본 구성 참고**: **HWDESC=0에서만 사용됩니다.** HWDESC=1이면 descriptor의 `dest_addr`가 쓰이고 이 레지스터 값은 무시됩니다 (무시 여부 실측은 § 11 #2 — SOR v2.0.0에도 대응 테스트 없음).

4.4.5 `SRC_ADDRESS` (0x414)

한 줄 요약: 레지스터 모드의 소스 주소. `DEST_ADDRESS`와 같은 규칙입니다.

구조·동작은 4.4.4와 같습니다 (`:193`, `:214`). 실측도 동일합니다.

4.4.6 `X_LENGTH` (0x418)

한 줄 요약: 레지스터 모드의 전송 길이. 값은 **"바이트 수 − 1"** 입니다. `0` = 1바이트, `0xFF_FFFF` = 16 MiB. **길이 0은 표현할 수 없습니다.**

Bits	Field	접근	리셋	설명
`[31:24]`	—	RO	`0`	쓰기 무시 (`DMA_LENGTH_WIDTH=24`)
`[23:0]`	`X_LENGTH`	RW	`0`	**전송 바이트 수 − 1**

```
        9'h106: up_dma_x_length[DMA_LENGTH_WIDTH-1:DMA_LENGTH_ALIGN]
                  <= up_wdata[DMA_LENGTH_WIDTH-1:DMA_LENGTH_ALIGN];
        //                    ^ DMA_LENGTH_ALIGN = 0 -> [23:0] 전부 쓰기 가능   :194
```

- **범위**: 1 B (`0x00_0000`) ~ 16 MiB (`0xFF_FFFF`).

"바이트 수 − 1"을 표로 보면 (descriptor의 `x_len` `+0x24`도 같은 규칙, § 5)

옮기고 싶은 바이트 수	써야 할 값	비고
1	`0`	가장 짧은 전송
256	`255`	SAN-004, SOR T-03
4,096	`4095`	SOR T-12
16 MiB	`0xFF_FFFF` (`0xFFFFFF`)	**가장 긴 전송** (24비트 모두 1)
0	—	**만들 수 없음.** 어떤 값을 써도 최소 1바이트

> ⚠️ **길이 0 전송은 이 형식으로 만들 수 없습니다.** 값 `0`이 이미 "1바이트"를 뜻하기 때문입니다. 24비트 필드가 표현하는 범위는 1바이트 ~ 16 MiB이고, 그 밖은 없습니다.
>
> **정정 메모 (v2.0.0).** v1.0.0 SOR의 시나리오 ET-005는 "길이 0 descriptor (`x_length` 필드가 `0xFFFFFF`)"라고 적었습니다. **이것은 틀렸습니다.** `0xFFFFFF`는 길이 0이 아니라 **최대 길이 16 MiB**입니다. SOR v2.0.0은 이 자리를 T-15의 "최대 길이(16 MiB) 전송"과 "자기 참조 체인"으로 바꾸었습니다 (`ERR-004`). 이 SFR v1.0.0 자체의 값(`0x00_0000` = 1 B, `0xFF_FFFF` = 16 MiB)은 처음부터 맞았고, 바뀌지 않았습니다.

- **본 구성 고유 동작**

구성	DMA_LENGTH_ALIGN	쓰기 가능 비트	길이 단위
ADI 표준 (Stream 포함)	3	`[23:3]` — 하위 3비트 상시 1	8 바이트

구성	DMA_LENGTH_ALIGN	쓰기 가능 비트	길이 단위
본 구성 (MM→MM)	0	[23:0] 전부	1 바이트

- **리셋값 주의**: upstream 문서는 `2**clog2(버스폭/8)-1` (= 7)로 적지만, 이는 정렬 비트가 있는 구성의 값입니다. **본 구성에서는 0**입니다. upstream 문서를 그대로 믿으면 안 되는 대표 사례이며, 이것이 D 등급을 따로 둔 이유입니다.
- **근거 R — 미측정**: 리셋값 `0x0000_0000`은 **RTL 분석 결과이며 실측되지 않았습니다.** upstream 문서는 `7`이라고 적고 있어 **두 값이 어긋난 상태**입니다. Sanity 범위 밖이고 SOR v2.0.0에도 이 값을 재는 테스트가 없으므로 검증팀이 확인해야 합니다 → §9.3. **본 문서에서 가장 우선순위가 높은 미측정 항목입니다.**
- **SOR 연결**: 1바이트 단위 길이(1, 7, 129 … 512바이트)의 실제 전송은 SOR **T-07** (`FUNC-005`, `FUNC-006`, `FUNC-007`, `AXI-002`, `AXI-003`)이 descriptor `x_len`으로 검증합니다. 최대 길이 `0xFFFFFF`는 **T-15** 3단계가 봅니다.

> **참고 — 0-replication 이슈와의 관계** `axi_dmac_regmap_request.v:140`에도 `{DMA_LENGTH_ALIGN{1'b1}}`이 있으나 **에러가 나지 않습니다.**
>
> ```
> reg [DMA_LENGTH_WIDTH-1:0] up_dma_x_length = {DMAC_DEF_X_LENGTH_DEFAULT[...], {DMA_LENGTH_ALIGN{1'b1}}};
> //                                            ^-- 다른 피연산자와 함께 concat 안 -> 합법
> ```
>
> Verilog-2001 LRM은 0-replication(0번 반복)을 **다른 피연산자가 있는 concatenation 안에서만** 허용합니다. `axi_dmac_burst_memory.v:163`은 단독으로 쓰여 불법이었고, 그것이 본 repo의 **유일한 RTL 패치** 근거입니다.

4.5 전송 상태 (0x428–0x430, 0x448–0x450)

4.5.1 `TRANSFER_DONE` (0x428)

한 줄 요약: 전송 번호(0~3)별 완료 표시등. **완료 판정에 쓰는 정식 레지스터**입니다(EOT 알림과 함께).

Bits	Field	접근	리셋	설명
[31]	PARTIAL_TRANSFER_DONE	RO	0	partial 정보가 FIFO에 남아 있음 (본 구성 Unreachable)
[30:4]	—	RO	0	예약
[3:0]	TRANSFER_n_DONE	RO	0	전송 ID별 완료 플래그

- **RTL 근거**: `axi_dmac_regmap_request.v:135`, `:364-380`, `:219`.
- **실측 (M)**: 리셋값 `0x0` (`SAN-003`). 전송 완료 후 비트 세트 확인(`SAN-004`).
- **동작 상세**
  - `up_sot` 시 `bit[TRANSFER_ID]` ← 0, `up_eot` 시 `bit[ACTIVE_TRANSFER_ID]` ← 1.
  - **ID를 재사용하기 전까지** 완료 상태를 유지합니다. 4개를 넘게 연속 제출하면 오래된 완료 정보가 덮어써집니다.
  - `ENABLE=0`에서 전부 0.
- **SG 모드 주의 — 두 카운터가 어긋날 수 있습니다.** 비트를 세우는 `up_eot`는 **EOT 플래그(`flags[1]`)가 설정된 descriptor**가 끝날 때마다 발생합니다. 반면 `TRANSFER_ID`는 **체인당** 1씩 증가합니다. 한 체인에 `flags[1]=1`인 descriptor가 2개 이상이면 1:1 대응이 깨집니다. 대응이 필요하면 **체인의 마지막 descriptor에만 `flags[1]`을 설정**하십시오 (§6.2).
- ⚠️ **드라이버/검증자 주의**: 알림을 막아 둔 설정에서 완료를 기다릴 때는 이 레지스터를 **반복해서 읽습니다** (SOR **T-09**).
- **SOR 연결**: "새로 출발하면 그 번호 비트가 0으로 지워짐"과 두 체인의 완료 구분은 **T-10** (`REG-013`, `IRQ-006`)이 처음 판정합니다. `SAN-004`는 "완료하면 비트가 선다"만 봅니다.

4.5.2 `ACTIVE_TRANSFER_ID` (0x42C)

한 줄 요약: 지금 진행 중인(다음에 완료될) 전송의 번호.

Bits	Field	접근	리셋	설명
[31:2]	—	RO	0	예약

Bits	Field	접근	리셋	설명
[1:0]	ACTIVE_TRANSFER_ID	RO	0	**다음에 완료될** 전송의 ID

- **RTL 근거**: axi_dmac_regmap_request.v:134, :375-377, :220.
- **폭 주의**: upstream 문서와 ADI TB 패키지는 [4:0]으로 적지만 **RTL은 2비트**입니다 (D 등급 불일치 사례).
- **근거 R — 미측정**: 리셋값 0x0000_0000은 RTL 분석 결과입니다. Sanity 범위 밖 → § 9.3.
- **SOR 연결**: 전송 중 값(ID_A)은 **T-10**, ENABLE=0 후 0은 **T-11**이 판정합니다. 둘 다 리셋 직후 값은 아닙니다.

4.5.3 STATUS (0x430)

한 줄 요약: 쓰이지 않는 예약 레지스터. 항상 0.

Bits	Field	접근	값	설명
[31:0]	RESERVED	RO	0	예약. 항상 0

- **RTL 근거**: axi_dmac_regmap_request.v:221 9'h10c: up_rdata <= 32'h0.
- 채널 상태는 DBG_STATUS(0x43C)를 사용합니다 (단, 관찰용 — § 4.7).

4.5.4 TRANSFER_PROGRESS (0x448)

한 줄 요약: 지금 전송에서 목적지까지 끝난 바이트 수. **전송 중에만 의미**가 있고, 완료 직후에는 0으로 지워집니다.

Bits	Field	접근	리셋	설명
[31:24]	—	RO	0	예약
[23:0]	TRANSFER_PROGRESS	RO	0	현재 전송에서 목적지 응답까지 끝난 누적 바이트 수

- **RTL 근거**: axi_dmac_regmap_request.v:153, :399-407, :222.
- **동작 상세**
  - 목적지 응답마다 measured_burst_length + 1을 더합니다.
  - **EOT가 실린 응답 다음 사이클에 0으로 지워집니다.** 따라서 **완료 직후에 읽으면 대개 0**이고, 전송 **도중**의 진행률 관찰에만 의미가 있습니다.
  - ENABLE=0에서 0.
- **SG 모드 주의**: 지움 조건의 EOT는 flags[1]=1인 descriptor에서만 발생합니다. 그러므로 **flags[1]=0인 descriptor의 바이트는 지워지지 않고 계속 누적**되어, EOT 플래그 descriptor까지의 **체인 누적 바이트**가 보입니다.
- ⚠️ **검증자 주의 — PAUSE 중 읽기**: PAUSE를 켠 직후에도 이미 나간 요청은 마무리되므로 값이 조금 더 늘 수 있습니다. 버스가 조용해진 **뒤에** 읽어야 값이 고정됩니다 (SOR **T-12** 4단계).
- **SOR 연결**: 전송 중 증가·PAUSE 중 고정·완료 후 0은 **T-12** (REG-018)가 처음 측정합니다. 체인 누적 동작은 T-12에서 "관측 후 확정"입니다.

> **명칭 주의 (결정 6 ⑤).** 선행 hrcho 문서는 이 레지스터를 TRANSFER_LENGTH로 적었으나, **완료 시 0으로 지워지는 동작**을 보면 TRANSFER_PROGRESS가 정확합니다. upstream 명칭을 채택했습니다. 현재 TB에는 이 레지스터 체크가 없어 **코드 반영 대상은 아닙니다.**

4.5.5 PARTIAL_TRANSFER_LENGTH (0x44C) / PARTIAL_TRANSFER_ID (0x450)

한 줄 요약: 스트림 소스가 일찍 끝났을 때의 기록. 본 구성에서는 쓸 일이 없습니다. **0x450은 읽기만 해도 내부 상태가 바뀌므로 절대 읽지 않습니다.**

레지스터	Bits	접근	설명
PARTIAL_TRANSFER_LENGTH	[23:0]	RO	partial FIFO head의 바이트 수
PARTIAL_TRANSFER_ID	[1:0]	**RO-RS**	partial FIFO head의 전송 ID. **읽으면 FIFO가 pop됨**

```
assign up_tlf_rd = up_rreq && up_raddr == 'h114;   // 0x450 읽기 -> pop   :382
```

- **읽기 순서**: 반드시 `LENGTH` → `ID` 순서로 읽습니다. `ID`를 먼저 읽으면 해당 항목의 길이를 잃습니다.
- **본 구성 — Unreachable**: partial 응답은 AXI-Stream 소스가 `X_LENGTH`보다 먼저 TLAST를 낼 때만 생깁니다. 소스가 AXI-MM이라 `response_partial`이 항상 0이므로 FIFO에 아무것도 들어가지 않습니다. 두 레지스터는 항상 0이고 `TRANSFER_DONE[31]`도 0입니다.

> ⚠️ **`0x450` 읽기 금지 — 레지스터 순회 테스트에서 제외합니다. "전 레지스터 덤프" 방식의 테스트는 `0x450`에서 내부 상태를 파괴합니다.** 본 구성에서는 FIFO가 비어 있어 실제 영향이 없지만, 구성이 바뀌면 바로 문제가 됩니다. Sanity `SAN-007`은 이 주소를 읽지 않습니다. SOR v2.0.0도 읽지 않습니다 (**T-02** 6단계에 명시). **읽기에 부작용이 있는 레지스터는 본 IP에서 이 하나뿐입니다.**

4.6 Scatter-Gather (0x454, 0x47C)

본 검증의 핵심 레지스터 2개입니다. 둘 다 `DMA_SG_TRANSFER = 1`일 때만 존재합니다.

4.6.1 `SG_ADDRESS` (0x47C)

한 줄 요약: 첫 descriptor(작업 지시서)가 있는 메모리 주소. 아래 3비트는 버려집니다.

Bits	Field	접근	리셋	설명
`[31:3]`	`SG_ADDRESS[31:3]`	RW	`0`	**첫 descriptor 주소**
`[2:0]`	—	RO	`0`	쓰기 무시, 읽기 0

- **RTL 근거**: `axi_dmac_regmap_request.v:333-354` (generate, `DMA_SG_TRANSFER=1`일 때만 존재), `:234`.
- **실측 (M)**: 리셋값 `0x0` (`SAN-003`). `0xFFFF_FFFF` 기입 → `0xFFFF_FFF8` (`SAN-007`).
- **정렬**: SG 인터페이스가 64비트이므로 **8바이트 정렬 필수**입니다. descriptor 하나는 48 B를 **6-beat burst 한 번**으로 읽으므로, **4 KiB 경계를 넘지 않도록 64바이트 정렬을 권장**합니다.
- **사용 범위**: 체인의 **첫 descriptor만** 이 레지스터로 지정합니다. 이후는 각 descriptor의 `next_sg_addr`에서 옵니다.
- **갱신 시점**: `TRANSFER_SUBMIT` 시점에 SG 엔진으로 넘어갑니다. 제출 후에는 다음 체인을 위해 바로 갱신해도 됩니다 (SOR **T-10** 연속 예약).
- ⚠️ **검증자 주의**: 레지스터가 아래 3비트를 버리는 것(`SAN-007`)과, 실제 descriptor **읽기 주소**가 맞춰지는 것은 별개입니다. 후자와 descriptor가 4 KiB 경계에 걸칠 때의 동작은 SOR **T-13** (`DESC-008`)이 봅니다 (경계 걸침은 "관측 후 확정").

4.6.2 `DESCRIPTOR_ID` (0x454)

한 줄 요약: SG 엔진이 처리 중인 descriptor의 `id` 값. **디버깅·추적용**이며 완료 판정에 쓰지 않습니다.

Bits	Field	접근	리셋	설명
`[31:0]`	`DESCRIPTOR_ID`	RO	`0`	SG 완료 추적 FIFO head의 descriptor `id`

```
assign fifo_in_data  = {hwdesc_flags & MASK_EOT_IRQ ? 1'b1 : 1'b0, hwdesc_id};
assign fifo_out_ready = resp_in_valid;            // <- 목적지 응답이 오면 pop
assign resp_out_id   = fifo_out_data[31:0];       // dmac_sg.v:312-315
```

- **동작 상세**
  - SG 엔진은 descriptor를 읽을 때마다 `{EOT 플래그, id}`를 **4-entry FIFO**에 넣고, 해당 descriptor의 전송이 끝날 때 하나씩 꺼냅니다.
  - 레지스터는 FIFO의 **head**를 조합 논리로 읽습니다. 전송 중에는 "현재 처리 중인 descriptor의 ID"입니다.

실측 (M) — 체인 완료 후의 값

본 구성에서는 체인 완료 후 `DESCRIPTOR_ID`가 마지막으로 완료된 descriptor의 id를 유지합니다.

테스트	체인	완료 후 읽은 값	해석
SAN-005	3개 (0xB0 → 0xB1 → 0xB2)	**0xB2**	자기 체인의 마지막 descriptor
SAN-006	1개 (0xC5)	**0xC5**	마지막 descriptor

FIFO가 비면 util_axis_fifo의 m_mem_read가 멈춰 출력이 갱신되지 않기 때문입니다. 선행 문서(jsha rev.001)는 "이전 전송의 stale 값이 보일 수 있음"으로 경고했는데, **SAN-005의 3-descriptor 체인이 앞선 SAN-004(0xA0)가 남긴 값이 아니라 0xB2를 반환했으므로 "이전 슬롯 잔류값"이 아님이 확인되었습니다.**

> ⚠️ **다만 이 동작은 ADI가 문서화한 계약이 아니라 util_axis_fifo의 구현 특성입니다.** 드라이버가 전송 완료 판정의 **근거로 삼는 것은 권장하지 않습니다.** 완료 확인은 EOT 인터럽트 또는 TRANSFER_DONE으로 하고, 이 레지스터는 **디버깅·추적 용도**로 씁니다. 검증 시나리오에서도 "마지막 id와 일치"를 **참고 체크만** 두고, 실패 시 IP 버그가 아니라 구현 변경 신호로 해석합니다. (SANITY v2.0.0은 SAN-005 #6, SAN-006 #7을 "구현 특성 체크"로, SOR v2.0.0은 완료 후 값을 항상 "참고 체크"로 둡니다.)

알림 시점에 읽으면 다음 장 id가 보일 수 있습니다 (v2.0.0 추가)

위 코드에서 FIFO는 **목적지 응답이 오는 순간 pop**됩니다(fifo_out_ready = resp_in_valid). EOT 알림(irq)은 그 **뒤에** 레지스터 경로를 거쳐 올라옵니다. 그래서 중간 장에 EOT 플래그를 켠 체인에서 **알림이 왔을 때 DESCRIPTOR_ID를 읽으면, 방금 끝난 장이 아니라 다음 장의 id가 보일 수 있습니다.**

상황	보일 수 있는 값	취급
전송 중 반복 읽기	처리 중인 장의 id. 순서대로 앞으로만 감 (건너뛰기 가능)	SOR **T-08**: 순서 판정 ("거꾸로 가면 FAIL")
중간 알림 때 읽기	끝난 장의 id **또는 다음 장의 id**	SOR **T-08**: 참고 체크, 관측 후 확정
마지막 장을 처리하는 동안	마지막 장의 id (예: 0xD7)	정상. "끝나기 전에는 마지막 id가 안 보여야 한다"는 기대는 틀림
체인 완료 후	마지막 장의 id (위 실측)	구현 특성 — 참고 체크

> ⚠️ **드라이버 주의**: 알림 핸들러에서 DESCRIPTOR_ID를 읽어 "몇 번째 장이 끝났다"고 판단하면 하나 앞서 갈 수 있습니다. 장 단위 진행 추적이 필요하면 **알림 횟수를 세거나 목적지 메모리를 확인**하십시오.

- **HWDESC=0이면** 0입니다 (RTL 분석. SOR **T-08** 5단계가 SG 모드를 끈 뒤 0x0000_0000을 참고 체크로 처음 봄).
- **SOR 연결**: **T-08** (DESC-006, REG-016) — 전송 중 추적과 위 표의 관측. T-03(0xA0), T-04(0xB2), T-06(마지막 장 id)은 완료 후 값을 참고 체크로만 봅니다.

4.6.3 SG 전용 동작 요약

항목	내용
활성 조건	CONTROL.HWDESC = 1
무시되는 레지스터	SRC_ADDRESS, DEST_ADDRESS, X_LENGTH (descriptor 값 사용)
그대로 쓰이는 레지스터	FLAGS.TLAST, IRQ_*, CONTROL
descriptor 읽기 버스	m_sg_axi (AR/R만 사용, ARLEN=5, ARSIZE=3, ARBURST=INCR)
descriptor 읽기 에러	**m_sg_axi_rresp를 검사하지 않음.** 에러 응답이 와도 데이터를 그대로 descriptor로 사용

> ⚠️ **RRESP 미검사는 설계상 한계이며 본 IP의 알려진 특성입니다.** SoC에서 descriptor 영역에 접근 권한 오류나 ECC 오류가 발생하면 **에러 표시 없이 쓰레기 주소로 전송이 진행**됩니다. 이 영역의 요구사항은 SOR v2.0.0 **T-15** (ERR-001 ~ ERR-003)가 흡수했으며, 현재 RTL 분석까지만 완료된 상태입니다. T-15는 동작이 정해지지 않은 테스트라 "hang 없음"만 판정하고 나머지는 관측 후 확정합니다.

4.7 디버그 (0x434–0x444)

`DISABLE_DEBUG_REGISTERS=0`이므로 모두 읽힙니다. **값의 타이밍이 버스 핸드셰이크에 묶여 있어 정확한 기대값 비교보다는 관찰용**입니다.

> ⚠️ **디버그 레지스터 규칙 (v2.0.0 — 이 문서와 SOR·SANITY 공통, 이곳이 기준)**
>
규칙	내용
1. 대상	`0x434` `CURRENT_DEST_ADDRESS`, `0x438` `CURRENT_SRC_ADDRESS`, `0x43C` `DBG_STATUS`, `0x440` `DBG_IDS0`, `0x444` `DBG_IDS1`
2. 용도	**관찰 기록만** 합니다. 값을 로그에 남기고, 동작을 이해하거나 디버깅할 때 씁니다
3. 금지	**합격 기준(판정)에 쓰지 않습니다.** 리셋값 비교에도 쓰지 않습니다
4. 이유	고정 리셋값이 없음 · 값이 버스 핸드셰이크 타이밍에 따라 달라짐 · `DBG_IDS0/1`은 Gray 인코딩
5. 읽기 부작용	없음. 전송 중 반복 읽어도 안전합니다
6. 함께 지킬 것	**`0x450`은 이 영역이 아니지만 읽기 금지입니다** (§ 4.5.5)
>
> 이 문서의 아래 절(4.7.1 ~ 4.7.4)에 적힌 "기대 패턴"은 **관찰할 때 참고할 예상값**이지 판정 기준이 아닙니다. SOR v2.0.0 2.1절은 이것을 "관찰 기록"이라고 부르고, SANITY v2.0.0 5.5절은 "기대값 비교 금지"로 적습니다. 셋은 같은 규칙입니다. 진행 상황을 판정해야 하면 `TRANSFER_PROGRESS`(§ 4.5.4)나 `TRANSFER_DONE`을 씁니다.

4.7.1 `CURRENT_DEST_ADDRESS` (0x434)

한 줄 요약: (관찰용) 목적지 쪽이 지금 쓰고 있거나 곧 쓸 burst의 시작 주소.

`m_dest_axi_awaddr[31:0]`을 그대로 읽습니다 (`axi_dmac_regmap.v:264`). burst 단위로 갱신되므로 "가장 최근에 발행했거나 발행 대기 중인 write burst의 시작 주소"로 해석합니다.

4.7.2 `CURRENT_SRC_ADDRESS` (0x438)

한 줄 요약: (관찰용) 소스 쪽이 지금 읽고 있거나 곧 읽을 burst의 시작 주소.

`m_src_axi_araddr[31:0]` (`:265`). 해석은 위와 같습니다.

> **명칭 주의 (결정 6 ⑧).** 선행 hrcho 문서는 `DBG_*_ADDR`로 적었으나 upstream 명칭은 `CURRENT_*_ADDRESS`입니다. 현재 TB에는 이 두 레지스터 체크가 없어 **코드 반영 대상은 아닙니다.**

- **SOR 연결**: 두 레지스터의 요구사항(`REG-017`)은 SOR v2.0.0에서 **제외**되었습니다. 디버그 레지스터라 값 비교를 하지 않는 규칙 대상이고, 값이 버스 타이밍에 따라 달라 정답을 정할 수 없기 때문입니다.

4.7.3 `DBG_STATUS` (0x43C)

한 줄 요약: (관찰용) 내부 reset manager(켜기·끄기·일시정지를 처리하는 상태 머신)의 상태와 도메인별 리셋·활성 비트.

upstream 레지스터 문서와 ADI TB 패키지에 **없는** 레지스터입니다. RTL에서만 확인됩니다.

Bits	Field	설명
`[31:12]`	—	0
`[11]`	`needs_reset`	ENABLE=0이 관측되어 리셋 시퀀스가 예약됨
`[10]`	`req_resetn`	request 도메인 리셋 해제(1)
`[9]`	`src_resetn`	src 도메인 리셋 해제(1)

Bits	Field	설명
[8]	dest_resetn	dest 도메인 리셋 해제(1)
[7]	sg_resetn	SG 도메인 리셋 해제(1)
[6]	req_enabled	request 도메인 활성
[5]	enabled_src	src 도메인 활성
[4]	enabled_dest	dest 도메인 활성
[3]	enabled_sg	SG 도메인 활성
[2:0]	state	reset manager 상태

state	이름	의미
0	DO_RESET	리셋 펄스 발생
1	RESET	리셋 완료 대기 / ENABLE=0 동안 머무름
2	DISABLED	리셋 완료, PAUSE 해제 대기
3	STARTUP	모든 도메인 활성화 대기
4	ENABLED	정상 동작
5	SHUTDOWN	진행 중 트랜잭션 마무리 대기

- **본 구성 참고**: 단일 클록·ASYNC_CLK_*=0 이므로 **4개 도메인 리셋이 같은 사이클에 풀리는 것이 정상**입니다. enabled_sg 는 HWDESC=0이면 활성 판정에서 제외됩니다.
- **정상 운용 시 예상 패턴 (관찰 참고값)**: 리셋 후 ENABLE=0 → state=1, ENABLE=1 → state=4, PAUSE=1 → state=2.
- **관찰 기록 (판정 아님) — v2.0.0에서 고침**: 이 레지스터는 **ENABLE / PAUSE 시퀀스 검증의 관측 창**으로 쓰면 유용합니다. 상태 머신 값이 직접 보이므로 "graceful tear-down(깔끔한 정지)이 SHUTDOWN 을 거쳤는가"를 **관찰**할 수 있습니다.
  - SOR v2.0.0 **T-11** (REG-024, v1.0.0 시나리오 FT-011): 동작 중 4, 조용해진 뒤 1을 예상. 중간의 5(마무리 중)는 짧아서 못 볼 수 있습니다.
  - SOR v2.0.0 **T-12** (REG-008, v1.0.0 시나리오 FT-012): 일시정지 중 2를 예상.
  - 두 경우 모두 **"관찰 기록"이며 합격 기준이 아닙니다.** v1.0.0은 이 값으로 "판정할 수 있다"고 적었으나, 디버그 레지스터 규칙(§4.7)과 충돌하므로 고쳤습니다. 끄기·일시정지의 판정은 버스 활동과 §8의 레지스터 값으로 합니다.
- **미측정 (R)**: 위 상태 값은 모두 RTL 분석입니다.

4.7.4 DBG_IDS0 (0x440) / DBG_IDS1 (0x444)

한 줄 요약: (관찰용) 목적지·소스 쪽 내부 burst 순번 태그. 값은 **Gray 코드**입니다.

```
DBG_IDS0 (dest)                          DBG_IDS1 (src)
 31  28 27  24 23 20 19 16 15 12 11  8 7  4 3  0     (각 byte: [7:4]=0 패딩, [3:0]=Gray ID)
+------+------+-----+-----+-----+-----+----+----+
| 0000 | resp | 0000| data| 0000| addr|0000| req|
+------+------+-----+-----+-----+-----+----+----+
```

레지스터	Bits	Field
DBG_IDS0	[27:24] / [19:16] / [11:8] / [3:0]	dest의 response / data / address / request ID
DBG_IDS1	동일	src의 response / data / address / request ID
공통	[31:28], [23:20], [15:12], [7:4]	0 (DBG_ID_PADDING=4)

- **값의 인코딩 — Gray**: 모든 필드는 **내부 burst 순번 태그의 Gray 코드 값**이며 **AXI transaction ID가 아닙니다.** 필드 하나가 burst 1개마다 inc_id() 로 Gray 순서로 전진합니다.
  - ENABLE=1 이후 누적 burst 수 N에서 정지 상태의 기대값 = b2g(N mod 16) (부록 B).
  - 두 필드의 거리(in-flight burst 수) = (g2b(x) − g2b(y)) mod 16. **Gray 값을 그대로 빼면 틀립니다.**

- **burst 수 계산**: `MAX_BYTES_PER_BURST=128`이므로 128 B 정렬 전송은 `ceil((X_LENGTH+1)/128)`개 burst입니다. SG descriptor fetch는 이 ID를 쓰지 않으므로 N에 포함되지 않습니다.
- **리셋**: `ENABLE=0` → 모든 ID 0.
- **Sanity·SOR 판정 대상에서 제외합니다.** Gray 인코딩 값이라 **안정적 기대값을 정의할 수 없고**, 인코딩 방식이 바뀌면 기대값도 바뀝니다. 읽기에 부작용은 없으므로 디버깅 참고용으로는 유효합니다.

> **Gray-ID 유지 결정(2026-10-01)과의 관계.** 단일 클록 구성에서 Gray 인코딩은 기능상 불필요하지만, **제거는 결함 수정이 아니라 설계 정책 판단**이므로 현 단계에서는 유지하기로 합의했습니다. 유지하는 한 이 두 레지스터의 값은 Gray이므로 위 해석이 필요합니다.

4.8 본 구성에서 비활성인 레지스터

한 줄 요약: 파라미터로 일부러 꺼 둔 레지스터·비트. **써도 바뀌지 않는 것이 정상**입니다.

파라미터(`generate`)로 저장소 자체가 만들어지지 않거나 쓰기가 마스크됩니다. **쓰기 무시, 읽기 0**입니다.

Offset	이름	비활성 조건	근거	실측
`0x41C`	`Y_LENGTH`	`DMA_2D_TRANSFER=0`	`regmap_request.v:271-275`	**M**
`0x420`	`DEST_STRIDE`	`DMA_2D_TRANSFER=0`	동일	R
`0x424`	`SRC_STRIDE`	`DMA_2D_TRANSFER=0`	동일	R
`0x458`	`FRAMELOCK_CONFIG`	`FRAMELOCK=0`	`:323-329`	**M**
`0x45C`	`FRAMELOCK_STRIDE`	`FRAMELOCK=0`	동일	R
`0x490` / `0x494` / `0x4BC`	주소 `*_HIGH`	**`ADDR_WIDTH=32`**	`if (HAS_ADDR_HIGH)` 가드	**M**
`0x400[3]`	`CONTROL.FRAMELOCK`	`FRAMELOCK=0`	`regmap.v:227`	**M**
`0x40C[0]`	`FLAGS.CYCLIC`	`CYCLIC=0`	`regmap_request.v:188`	**M**

> **이 표의 목적은 버그 검출이 아니라 오판 방지입니다.** 이 동작을 모르는 사람이 나중에 "RW 레지스터 write / read-back" 테스트를 작성하면 **정상 동작을 버그로 보고**합니다. 전 비트 1을 쓴 뒤 0이 읽히는 것을 확인하면 반대로 **"파라미터로 기능이 빠졌음"을 레지스터 수준에서 증명**할 수 있습니다. Sanity `SAN-007`과 SOR v2.0.0 **T-02** (`REG-021`, `REG-022`)가 이 목적의 테스트입니다. T-02는 R로 남은 `0x420`, `0x424`, `0x45C`의 값 `0`도 처음 측정합니다 (리셋 직후 시점은 아님).

---

5. SG 디스크립터 메모리 포맷

`CONTROL.HWDESC=1`에서 SG 엔진(`dmac_sg.v`)이 `m_sg_axi`로 읽는 메모리 구조입니다. **SG 기능의 실체는 레지스터가 아니라 이 포맷에 있습니다.**

비유하면 **작업 지시서 한 장의 양식**입니다. 한 장에 "어디서 읽고(src), 어디에 쓰고(dest), 몇 바이트이고(x_len), 다음 장은 어디 있다(next)"가 적혀 있습니다. CPU가 이 양식대로 메모리에 적어 두면, DMA가 한 장씩 읽어 가며 일합니다.

- **크기**: 48바이트 (64비트 × 6 beat, `ARLEN=5`)
- **정렬**: 8바이트 필수, **64바이트 권장** (한 burst가 4 KiB 경계를 넘지 않도록)
- **엔디언**: little-endian

Offset	크기	Field	비트 사용	본 구성 처리	beat
`+0x00`	32	`flags`	`[0]` LAST, `[1]` EOT_IRQ, `[31:2]` 무시	사용	0 `[31:0]`
`+0x04`	32	`id`	`[31:0]`	사용 → `DESCRIPTOR_ID`	0 `[63:32]`
`+0x08`	64	`dest_addr`	`[63:3]`	사용 (상위 32b는 `ADDR_WIDTH=32`에서 무시)	1

Offset	크기	Field	비트 사용	본 구성 처리	beat
`+0x10`	64	`src_addr`	`[63:3]`	사용 (동일)	2
`+0x18`	64	`next_sg_addr`	`[63:3]`	`flags[0]=0` 일 때 다음 descriptor	3
`+0x20`	32	`y_len`	—	**무시** (2D 비활성) — 0 권장	4 `[31:0]`
`+0x24`	32	`x_len`	`[23:0]` (바이트 − 1)	사용	4 `[63:32]`
`+0x28`	32	`src_stride`	—	**무시** — 0 권장	5 `[31:0]`
`+0x2C`	32	`dst_stride`	—	**무시** — 0 권장	5 `[63:32]`

`flags` 상세

Bit	이름	의미
`[0]`	`LAST`	1 = 체인의 마지막. 처리 후 SG 엔진이 IDLE로 복귀. 0 = `next_sg_addr` 를 이어서 읽음
`[1]`	`EOT_IRQ`	1 = 이 descriptor의 데이터 전송이 끝나면 `TRANSFER_COMPLETED` 이벤트 발생

자주 쓰는 `flags` 값: 마지막 장 `0x3` (LAST + EOT_IRQ), 중간 장 `0x0`. 상세는 § 6.2.

`x_len` (`+0x24`) — "바이트 수 − 1" (v2.0.0에서 명확히 함)

`x_len` 은 레지스터 `X_LENGTH` (§ 4.4.6)와 같은 규칙입니다.

x_len 값	옮기는 바이트 수
`0`	**1바이트** (가장 짧음)
`255`	256바이트
`0xFFFFFF`	**16 MiB** (가장 긺)
—	**길이 0은 만들 수 없음**

> ⚠ **길이 0 descriptor는 이 형식으로 만들 수 없습니다.** `x_len = 0` 은 이미 1바이트입니다. v1.0.0 SOR ET-005의 "길이 0 descriptor (`x_length` 필드가 `0xFFFFFF`)"는 틀린 표현이었습니다. `0xFFFFFF` 는 **16 MiB 최대 길이**입니다. SOR v2.0.0 **T-15** (`ERR-004`)는 그 자리에서 16 MiB 전송(첫 쓰기 확인 후 `ENABLE=0` 으로 끊음)과 자기 참조 체인을 봅니다.

> ⚠ **주의 1 — `x_len` 상위 비트.** descriptor는 32비트지만 내부 레지스터가 24비트라 `[31:24]` 가 **조용히 잘립니다.** 16 MiB 이상을 한 descriptor에 쓰면 길이가 틀어집니다.
>
> ⚠ **주의 2 — 끝나지 않는 체인.** `LAST` 가 하나도 없는 순환 체인(예: "다음 장 주소"가 자기 자신)은 합법적인 "HW cyclic" 구성입니다. **`ENABLE=0` 으로만 멈출 수 있습니다.** `DMA_CYCLIC=0` 이어도 이 경로는 막히지 않습니다.
>
> ⚠ **주의 3 — 읽기 에러 미검출.** `RRESP` 를 검사하지 않으므로, 잘못된 주소의 descriptor를 읽어도 **에러 표시 없이 쓰레기값으로 전송이 진행**될 수 있습니다.

주의 2와 "hang 없음"의 관계 (v2.0.0 추가)

자기 참조 체인과 16 MiB 전송은 **원래 시간 안에 스스로 끝나지 않습니다.** 이것은 이 문서 기준으로 **정상 동작**입니다. 그래서 "시간 안에 끝나지 않음 = hang(멈춤)"으로 판정하면 정상 동작을 버그로 보고하게 됩니다.

SOR v2.0.0 2.3절은 "hang 없음"을 아래처럼 정의합니다. 이 문서도 같은 정의를 씁니다.

#	조건	판정
1	시간 안에 스스로 끝남	hang 아님
2	끝나지 않더라도, `CONTROL ← 0x4` (켜기만 끄기)를 쓴 뒤 **(가)** 버스가 50,000 클럭 안에 idle(쉬는 상태)로 돌아오고, **(나)** `CONTROL ← 0x5` 로 다시 켜서 보낸 T-03 전송(descriptor 1장, 256바이트)이 정상으로 끝남	hang 아님
3	(가) 또는 (나)가 실패	**hang**

이 정의는 §4.3.1의 `ENABLE` 동작(진행 중 트랜잭션을 마무리한 뒤 리셋 펄스)과 §8의 매트릭스에 기대고 있습니다. 즉 **"`ENABLE=0`으로 빠져나올 수 있는가"** 가 hang의 기준입니다. 주의 3(읽기 에러)으로 descriptor가 0으로 읽혀 주소 0을 계속 따라가는 경우도 같은 기준으로 판정합니다 (SOR **T-15** 2단계).

실측 (M): Sanity `SAN-004`(1개) / `SAN-005`(3개 체인) / `SAN-006`(1개)에서 위 포맷으로 descriptor를 구성해 정상 동작을 확인했습니다. `SAN-005` 로그에서 **체인 3개가 첫 데이터 전송 전에 모두 prefetch**(미리 읽기)되는 것도 관측되었습니다 (SOR **T-04** (`DESC-009`)에서 판정).

SOR 연결: 포맷 해석 **T-03** (`DESC-001` ~ `DESC-003`, `DESC-007`), 체인 따라가기 **T-04** (`DESC-004`), 체인 길이 **T-06**, 바이트 단위 `x_len` **T-07**, EOT_IRQ 중간 장 **T-08** (`DESC-005`), 정렬·4 KiB 경계 **T-13** (`DESC-008`), 에러·극단값 **T-15** (`ERR-001` ~ `ERR-004`).

---

6. 인터럽트 동작

6.1 신호 경로

```
                +------------------- IRQ_SOURCE (0x088) --------------------+
up_sot --+      |  source <= {eot, sot} | (source & ~clear)                |
up_eot --+----> |  clear  = (write to 0x084) ? wdata[1:0] : 0               |
                +---------------------------+-------------------------------+
                                            |
                     IRQ_MASK (0x080) ---> ~mask &
                                            |
                                  IRQ_PENDING (0x084)
                                            |
                                    OR-reduce, 1 FF
                                            |
                                          irq   (level, active-high)
```

- `irq`는 `IRQ_PENDING` 비트 OR를 **1사이클 레지스터**로 낸 **레벨 신호**입니다.
- 원인을 지울 때까지 유지됩니다. **엣지 트리거 인터럽트 컨트롤러에 연결하면** 한 이벤트 처리 중 다른 이벤트가 들어올 때 레벨이 내려가지 않아 두 번째 엣지를 놓칠 수 있습니다. **레벨 트리거로 연결**합니다.
- **실측 (M)**: W1C 후 `irq` de-assert 확인 — `SAN-006` check 6.

6.2 SG 모드(HWDESC=1)에서의 이벤트 의미

이벤트	HWDESC=0	HWDESC=1 (본 구성)
`TRANSFER_QUEUED` (`up_sot`)	레지스터 요청 수락 시 1회	**체인 제출 1회당 1회**
`TRANSFER_COMPLETED` (`up_eot`)	전송 완료 시 1회	**`flags[1]=1`인 descriptor가 끝날 때마다** 1회

권장 descriptor 플래그 설정

목적	설정	결과
체인당 완료 인터럽트 1회 (기본)	마지막만 `flags = 0x3`, 나머지 `0x0`	`TRANSFER_ID` ↔ `TRANSFER_DONE` 1:1, 인터럽트 최소화
descriptor별 진행 추적	모두 `flags[1]=1`, 마지막만 `flags[0]=1`	descriptor마다 EOT. **`TRANSFER_DONE`/`ACTIVE_TRANSFER_ID`가 체인 ID와 어긋남**
인터럽트 없는 체인	모두 `flags[1]=0`	EOT 없음. **`TRANSFER_DONE` 미세트, `TRANSFER_PROGRESS` 미클리어** — 완료 확인은 메모리 polling만 가능

Sanity `SAN-005`가 첫 번째 방식(마지막만 `0x3`)을 사용합니다. SOR v2.0.0에서는 첫 번째 방식을 **T-04** ("`irq` 1번만")가, 중간 장에도 EOT를 켜는 경우를 **T-08** (`DESC-005`, `IRQ-001`, v1.0.0 시나리오 `FT-006`)이 확인합니다.

⚠️ **중간 알림이 너무 가까우면 하나로 보일 수 있습니다.** `irq`는 레벨 신호이므로, 앞 알림을 지우기 전에 다음 EOT가 오면 `irq`가 내려가지 않습니다. 알림 횟수를 셀 때는 이 점을 고려합니다 (SOR T-08은 그런 실행을 "무효"로 기록하고 다시 돌림).

6.3 인터럽트 서비스 루틴 권장 순서

1. `IRQ_PENDING`(0x084) 읽기 → 처리할 비트 확인
2. **같은 값을 `IRQ_PENDING`에 써서 clear** (RW1C)
3. `TRANSFER_COMPLETED`(비트1)이면 `TRANSFER_DONE`(0x428)으로 완료된 전송 ID 확인
4. `TRANSFER_QUEUED`(비트0)이면 다음 전송 제출 가능 — **완료가 아닙니다** (아래 주의)

clear(2)를 상태 확인(3)보다 먼저 하는 이유: clear 이후 들어온 이벤트는 "이벤트 우선" 규칙에 따라 다시 래치되므로 잃지 않습니다. 반대로 상태 확인 후 clear하면 **그 사이에 들어온 이벤트를 함께 지워버릴 수 있습니다.**

⚠️ **드라이버 주의 — 비트0만 올라온 알림은 완료가 아닙니다.** `IRQ_MASK`에서 비트0(접수)을 허용했다면(`0x0` 또는 `0x2`), 출발 직후 비트0만 선 알림이 먼저 옵니다. 이때 목적지 데이터를 읽거나 버퍼를 해제하면 **아직 옮겨지지 않은 데이터를 쓰게 됩니다.** 완료 처리는 반드시 **비트1** 또는 `TRANSFER_DONE`을 확인한 뒤에 합니다 → § 6.4.

6.4 `IRQ_MASK` = 0x0일 때 먼저 오는 접수 알림

v2.0.0에서 새로 넣은 절입니다 (통합 문서 7장 #8, SOR v2.0.0 T-09).

`IRQ_MASK = 0x0`은 "접수(비트0)와 완료(비트1) 알림을 **둘 다** 받겠다"는 뜻입니다. 이때 한 번의 전송에서 일어나는 일은 아래 순서입니다.

순서	사건	IRQ_PENDING	irq	의미
1	`TRANSFER_SUBMIT ← 0x1` 후 요청이 수락됨 (`up_sot`)	`0x1`	**1**	**접수.** 데이터는 아직 안 옮겨짐
2	(드라이버가 비트0을 지우지 않았다면) 데이터 전송 진행	`0x1`	1	—
3	마지막 장 전송 완료 (`up_eot`)	`0x3`	1	**완료**

⚠️ **첫 `irq`는 완료가 아니라 접수입니다.** `irq`가 올라오면 바로 "끝났다"고 처리하는 드라이버나 테스트는 `IRQ_MASK = 0x0`에서 **틀린 시점에 완료 처리**를 합니다.

- 드라이버: `IRQ_PENDING` 비트를 확인하고, 비트1이 설 때까지 완료 처리를 하지 않습니다.
- 검증: 마스크 조합 시험에서는 `irq`가 아니라 **`TRANSFER_DONE`을 반복해서 읽어** 완료를 기다립니다 (SOR **T-09**).
- 표준 순서(§ 7.2)는 `IRQ_MASK = 0x1`(접수는 막고 완료만 허용)을 써서 이 혼동을 피합니다.

참고로 마스크 네 가지에서 전송이 끝난 뒤의 기대값은 아래와 같습니다 (RTL 동작 § 4.2로부터, SOR T-09 통과 기준과 같음).

IRQ_MASK	올라오는 irq	IRQ_PENDING	IRQ_SOURCE
`0x3`	없음	`0x0`	`0x3`
`0x2`	"접수" 알림만	`0x1`	`0x3`
`0x1`	"완료" 알림만	`0x2`	`0x3`
`0x0`	둘 다 (출발 직후 "접수"가 먼저)	`0x3`	`0x3`

(`IRQ_SOURCE`는 마스크와 무관하게 두 사건을 모두 기록하므로 항상 `0x3`입니다. 실측은 `0x1` 한 가지만 — `SAN-006`.)

7. 프로그래밍 시퀀스

모든 접근은 **32비트 단위**입니다.

7.1 초기화 및 sanity check

#	동작	기대값	Sanity TC
1	`R(0x000)` VERSION	`0x0004_0565`	`SAN-001`
2	`R(0x00C)` IDENTIFICATION	`0x444D_4143`	`SAN-001`
3	`R(0x010)` INTERFACE_DESCRIPTION_1	`0x3807_0303`	`SAN-001`
4	`R(0x014)` INTERFACE_DESCRIPTION_2	`0x0000_0000`	`SAN-001`
5	`W(0x008, 0xA5A5_5A5A)` / `R(0x008)`	기입값	`SAN-002`
6	`R(0x400)` CONTROL	`0x0000_0000`	`SAN-003`
7	`R(0x080)` IRQ_MASK	`0x0000_0003`	`SAN-003`
8	`R(0x40C)` FLAGS	**`0x0000_0002`**	`SAN-003`

1번이나 2번이 틀리면 버스·디코딩 문제이므로 그 뒤 단계를 진행하지 않습니다.

7.2 SG 전송 실행 (본 구성 표준 흐름)

SOR v2.0.0 1.2절의 "표준 순서"와 같습니다.

```
1. 메모리에 descriptor 체인 작성 (§5 포맷, 8B 정렬)
     마지막 descriptor만 flags = 0x3 (LAST | EOT_IRQ)

2. W(0x080, 0x1)      // IRQ_MASK — SOT 마스크, EOT 허용
3. W(0x400, 0x5)      // CONTROL  — ENABLE | HWDESC
4. W(0x47C, <첫 descriptor 주소>)
5. W(0x408, 0x1)      // TRANSFER_SUBMIT

6. irq 대기  (또는 R(0x428) TRANSFER_DONE polling)

7. R(0x084)           // IRQ_PENDING
8. W(0x084, <7의 값>) // W1C — IRQ_SOURCE 클리어
9. R(0x428)           // 완료된 전송 ID 확인
```

> ⚠️ **3번과 5번의 순서는 바꿀 수 없습니다.** `ENABLE=0` 상태에서 `TRANSFER_SUBMIT`을 쓰면 **아무 일도 일어나지 않고 에러도 나지 않습니다**(§4.4.2). 조용히 실패하므로 디버깅이 어렵습니다. SOR **T-10** (`REG-011`) 1단계가 이 동작을 확인합니다.

> ⚠️ **2번의 `0x1`을 `0x0`으로 바꾸면 6번의 첫 `irq`는 "접수"입니다.** 완료가 아닙니다. 이 경우 7번에서 읽은 값의 **비트1**을 확인한 뒤에만 완료로 처리하십시오 (§6.4).

> ⚠️ **6번에서 `irq`를 기다릴 수 있는 것은 2번에서 완료 알림(비트1)을 허용했을 때뿐입니다.** `IRQ_MASK`를 `0x3`이나 `0x2`로 두면 완료 `irq`가 오지 않습니다. 그때는 `TRANSFER_DONE`을 반복해서 읽습니다.

7.3 연속 전송

#	동작	비고
1	`R(0x408)`이 0인지 확인	0이 아니면 이전 요청 미수락

#	동작	비고
2	`R(0x404)` TRANSFER_ID 기록	이번 전송에 부여될 ID
3	`W(0x47C, <다음 체인 주소>)`	
4	`W(0x408, 0x1)`	

최대 4개까지 추적 가능합니다(`TRANSFER_DONE[3:0]`). 그 이상 연속 제출하면 오래된 완료 정보가 덮어써집니다. SOR v2.0.0 **T-10**(`REG-012` ~ `REG-014`, `FUNC-008`, `IRQ-006`)이 체인 A가 도는 동안 체인 B를 예약하는 이 흐름을 검증합니다.

---

8. 리셋 · ENABLE · PAUSE 영향 매트릭스

한 줄 요약: HW 리셋은 전부 지우고, `ENABLE=0`은 진행 상태만 지우고, `PAUSE=1`은 아무것도 지우지 않습니다.

> ⚠️ **이 표의 근거는 전부 R(RTL 분석)이며 아직 측정한 적이 없습니다.** (HW 리셋 열 중 부록 A에서 M인 칸 제외) SOR v2.0.0의 **T-11**(`ENABLE=0` 열)과 **T-12**(`PAUSE=1` 열)이 이 표를 **처음으로 측정**합니다. 측정 결과가 표와 다르면, RTL 버그인지 이 표가 틀렸는지를 **먼저 가립니다.** 표가 틀렸다면 이 문서를 고칩니다.

레지스터 / 상태	HW 리셋	ENABLE=0	PAUSE=1
`SCRATCH`	0	유지	유지
`IRQ_MASK`	`0x3`	유지	유지
`IRQ_SOURCE`	0	**유지**	유지
`IRQ_PENDING` (조합)	0	SOURCE 따라감	유지
`CONTROL`	0	—	—
`FLAGS`	`0x2`	유지	유지
`DEST/SRC_ADDRESS`	0	유지	유지
`X_LENGTH`	0	유지	유지
`SG_ADDRESS`	0	유지	유지
`TRANSFER_ID`	0	**0**	유지
`ACTIVE_TRANSFER_ID`	0	**0**	유지
`TRANSFER_DONE`	0	**0**	유지
`TRANSFER_SUBMIT`	0	**0**	유지
`TRANSFER_PROGRESS`	0	**0**	유지
partial FIFO	비움	**비움**	유지
DMA 데이터 경로	리셋	**리셋 펄스**	정지(상태 보존)
`DBG_IDS0/1`	0	**0**	유지

핵심: `ENABLE=0`은 **설정은 남기고 진행 상태만 지웁니다.** 테스트 간 상태 격리에 쓸 수 있지만, 의도치 않게 내리면 진행 중이던 전송 정보를 잃습니다.

SOR v2.0.0과의 연결

열	측정하는 테스트 (원래 ID)	판정하는 행	비고
`ENABLE=0`	**T-11** (`REG-006`, `REG-024`; v1.0.0 시나리오 `FT-011`)	`TRANSFER_ID`, `ACTIVE_TRANSFER_ID`, `TRANSFER_DONE`, `TRANSFER_PROGRESS` = 0 / `SG_ADDRESS`, `FLAGS`, `IRQ_MASK`, `IRQ_SOURCE` = 끄기 전 값 그대로	쓰는 값은 `CONTROL ← 0x4`(켜기만 끔). `DBG_IDS0/1` 행은 디버그 레지스터라 판정하지 않음

열	측정하는 테스트 (원래 ID)	판정하는 행	비고
`PAUSE=1`	**T-12** (`REG-008`, `REG-018`; v1.0.0 시나리오 `FT-012`)	추적 값(`TRANSFER_ID` 등)이 지워지지 않음, 멈춘 자리부터 이어 감	쓰는 값은 `CONTROL ← 0x7`. `TRANSFER_PROGRESS`는 **버스가 조용해진 뒤** 읽음 (§ 4.5.4)

이 표 자체가 원래 요구사항 `REG-006`·`REG-008`의 기대값 정의입니다. 위 두 테스트가 ENABLE / PAUSE 조작 후 이 열을 그대로 검증합니다. 표의 한 칸이라도 틀리면 T-11 또는 T-12가 FAIL합니다 — 그것이 이 테스트들의 가치입니다.

> ⚠️ **`PAUSE`를 켠 직후 바로 읽지 마십시오.** 이미 나간 버스 요청은 `PAUSE` 뒤에도 마무리되므로, `TRANSFER_PROGRESS` 같은 값이 잠깐 더 바뀔 수 있습니다. "유지" 칸은 **세 버스가 조용해진 뒤**의 값을 뜻합니다.

---

9. Sanity Test 연계

> 레지스터 맵과 회귀 테스트가 어긋나지 않도록 매핑을 명시합니다. 상세 구현은 `SGDMA-SANITY-rel-v2.0.0`에 있습니다. SOR v2.0.0 연계는 § 9.4에 있습니다.

항목	값
구현	`tb/sg_dma/tb_sg_dma.sv`
구성	**TC 7건 / 체크 46건**
최종 실행	**2026-10-06 — 46/46 PASS**
범위	**회귀 게이트로 한정 (2026-10-06 팀 결정)** — 부록 D.3
v2.0.0	SANITY v2.0.0은 **문서만** 고쳤고 TB·체크·기대값은 v1.0.0과 같습니다

9.1 TC ↔ 레지스터 매핑

TC	체크	검증 레지스터	주로 연결되는 SOR v2.0.0 테스트
`SAN-001` Register identity	5	`0x000`, `0x004`, `0x00C`, `0x010`, `0x014`	T-01
`SAN-002` Scratch R/W	3	`0x008`	T-01
`SAN-003` Reset defaults	10	`0x400`, `0x080`, `0x084`, `0x088`, `0x40C`, `0x428`, `0x404`, `0x410`, `0x414`, `0x47C`	T-09 (`IRQ_MASK`만)
`SAN-007` Parameter-masked bits	11	`0x400[3]`, `0x40C[0]`, `0x410/414/47C[2:0]`, `0x490/494/4BC`, `0x41C`, `0x458`, 미매핑	T-02
`SAN-004` Single descriptor	4	`0x428` + 데이터 경로	T-03
`SAN-005` 3-descriptor gather	6	`0x454` + 데이터 경로	T-04
`SAN-006` IRQ & status	7	`0x080`, `0x084`, `0x088`, `0x454`, `irq`	T-09

실행 순서: `SAN-001 → 002 → 003 → 007 → 004 → 005 → 006`

레지스터 계층을 먼저 돌리는 이유는 **버스가 죽은 상태에서 전송 시나리오를 돌리면 원인 분리가 안 되기** 때문입니다. `SAN-003`은 **해당 레지스터에 쓰기가 발생하기 전에** 실행되어야 하므로 `SAN-007`(전 비트 1 기입)보다 앞에 있습니다.

> **`DESCRIPTOR_ID` 체크 2개(`SAN-005 #6 = 0xB2`, `SAN-006 #7 = 0xC5`)는 구현 특성 체크입니다** (§ 4.6.2). TB에서는 다른 체크와 똑같이 집계되지만, FAIL하면 RTL 버그보다 "구현이 바뀌었다"는 신호로 먼저 봅니다 (SANITY v2.0.0 3.8절).

9.2 오판 방지 체크 (SAN-007)

파라미터로 **의도적으로 죽인 비트**를 버그로 오인하지 않도록, 기대값을 문서와 TB 양쪽에 고정했습니다.

동작	기대값	근거
W(0x400, 0xFFFF_FFFF) → R	0x0000_0007	FRAMELOCK=0
W(0x40C, 0xFFFF_FFFF) → R	0x0000_0006	CYCLIC=0
W(0x410, 0xFFFF_FFFF) → R	0xFFFF_FFF8	8B 정렬 (0x414/0x47C 동일)
R(0x490/494/4BC)	0x0000_0000	ADDR_WIDTH=32
R(0x41C), R(0x458)	0x0000_0000	2D=0, FRAMELOCK=0
R(0x0F0), R(0x600)	0x0000_0000	미매핑, 에러 응답 없음

SAN-007은 **건드린 레지스터를 모두 원복한 뒤 반환**합니다. 원복하지 않으면 이후 TC의 전제가 깨집니다. SOR v2.0.0 **T-02**는 같은 동작에 더해 0x420·0x424·0x45C 읽기, 읽기 전용 레지스터 쓰기(REG-019), WSTRB 무시(REG-023)를 봅니다.

9.3 Sanity 미커버 영역

> **Sanity는 회귀 게이트이지 커버리지 도구가 아닙니다.** 아래 항목은 **의도적으로 검증팀에 남긴 것**이며, 누락이 아닙니다. 본 문서에서 근거 등급이 **R**로 남아 있는 값들이 대부분 여기에 해당합니다. 오른쪽 열은 SANITY v2.0.0 5.2·5.3절과 같은 내용입니다.

리셋값 미측정 (근거 R)

"SOR v2.0.0에서" 열은 SOR v2.0.0이 그 레지스터를 **어떤 형태로라도** 읽는지입니다. **"리셋 직후 값"을 판정하는 SOR 테스트는 없습니다.**

레지스터	상태	비고	SOR v2.0.0에서
0x418 X_LENGTH	**R — 최우선**	**upstream 문서는 7, RTL 분석은 0. 두 값이 어긋난 채 미측정**	**어디에도 없음**
0x408 TRANSFER_SUBMIT	R		T-10이 ENABLE=0에서 쓴 뒤 다시 읽음 (리셋값 측정은 아님)
0x42C ACTIVE_TRANSFER_ID	R		T-10, T-11이 전송 중·끈 뒤 값을 판정
0x420 / 0x424 *_STRIDE	R	비활성 레지스터	T-02가 값 0을 처음 측정 (리셋 직후 시점은 아님)
0x430 STATUS	R	상시 0	없음
0x448 TRANSFER_PROGRESS	R		T-12가 전송 중·끝난 뒤 값을 판정
0x44C PARTIAL_TRANSFER_LENGTH	R	본 구성 Unreachable	없음
0x454 DESCRIPTOR_ID	R	**리셋 직후 값만** 미측정 (전송 후 값은 M — 구현 특성)	T-08이 SG 모드를 끈 뒤 0을 참고 체크
0x45C FRAMELOCK_STRIDE	R		T-02가 값 0을 처음 측정 (리셋 직후 시점은 아님)
0x498 / 0x49C CURRENT_*_HIGH	R		없음
0x450 PARTIAL_TRANSFER_ID	**측정 금지**	**읽기가 FIFO를 pop** → § 4.5.5	**읽기 금지** (T-02 6단계)
0x434 ~ 0x444 디버그군	**비교 제외**	고정 리셋값 없음 → § 4.7	**관찰 기록만** (T-11, T-12가 DBG_STATUS 기록)

기능 미커버

영역	필요한 것	맡은 SOR v2.0.0 테스트 (원래 ID)
`CONTROL` ENABLE / PAUSE 토글 효과	§8 매트릭스 전체 검증	**T-11** (`REG-006`, `REG-024`), **T-12** (`REG-008`)
`TRANSFER_SUBMIT` self-clear 타이밍	전송 중 관측	**T-10** (`REG-009` ~ `REG-011`)
`TRANSFER_PROGRESS` 누적·클리어	**전송 중** 관측 (완료 후엔 0)	**T-12** (`REG-018`)
디버그군 (`0x434`~`0x444`)	전송 중 관측	`DBG_STATUS`는 T-11·T-12에서 **관찰 기록만**. `CURRENT_*_ADDRESS`는 **SOR에서도 제외** (`REG-017`)
SG 모드 시 `0x410`/`0x414`/`0x418` 무시 여부	§11 #2	**없음** — SOR v2.0.0 15개 테스트에 대응 시험 없음
scatter, 체인 길이, 바이트 단위 길이, 정렬·4 KiB 경계, 느린 버스·`AxCACHE` 출력, 에러 주입	레지스터 맵 밖의 기능	T-05, T-06, T-07, T-13, T-14, T-15 (상세는 SANITY v2.0.0 5.3절)

리셋값 미측정 항목의 처리 (2026-10-06 결정)

위 리셋값 항목들은 Sanity에도 SOR에도 포함되지 않습니다. SOR의 3-1 Reset / Clock이 "Sanity Test에서 커버됨 · 해당 사항 없음"으로 정리되고 `RST` 요구사항 6건이 삭제되었으므로, **현 시점에서 수행 계획이 없습니다.** SOR v2.0.0도 이 결정을 그대로 따릅니다 (v1.0.0의 ET-004 "전송 중 리셋"도 넣지 않음).

따라서 이 항목들은 **근거 등급 R인 채로 인계**됩니다. 특히 `X_LENGTH`(0x418)는 upstream 문서(`7`)와 RTL 분석(`0`)이 어긋난 상태이며, **어느 쪽이 맞는지 아무도 확인하지 않았습니다.**

> **나중에 측정하게 된다면** — 개별 추가보다 부록 A 전체를 읽는 sweep 한 건이 누락 없이 닫습니다. 그때 **반드시 지켜야 할 제약이 둘** 있습니다.
>
> 1. **`0x450`을 읽지 말 것** — 읽기가 내부 FIFO를 pop합니다
> 2. **`0x434`~`0x444` 디버그군 제외** — 고정 리셋값이 없어 기대값을 정의할 수 없습니다
>
> 전수 덤프 방식으로 순진하게 작성하면 1번에서 내부 상태를 파괴합니다. **수행 계획이 없더라도 이 두 줄은 지우지 않습니다** — 나중에 누가 하든 같은 함정에 빠지기 때문입니다.

9.4 SOR v2.0.0 테스트 ↔ 레지스터 대응

v2.0.0에서 새로 넣은 절입니다. SOR v2.0.0의 테스트 15개가 이 문서의 어느 레지스터·절을 기대값의 근거로 쓰는지 정리했습니다. 괄호 안은 흡수된 원래 요구사항 ID입니다. 판정 종류(판정 / 참고 체크 / 관찰 기록 / 관측 후 확정)의 뜻은 SOR v2.0.0 2.1절을 따릅니다.

테스트	제목	주로 쓰는 레지스터	근거 절	이 문서 기준 주의
T-01	레지스터 버스와 기본 구성 확인 (`REG-001` ~ `REG-005`, `REG-020`)	`0x000`, `0x004`, `0x008`, `0x00C`, `0x010`, `0x014`	§4.1	`VERSION`/`IDENTIFICATION`이 틀리면 나머지 건너뜀
T-02	일부러 꺼 둔 비트와 레지스터 접근 규칙 (`REG-019`, `REG-021` ~ `REG-023`)	`0x400`, `0x40C`, `0x410`/`0x414`/`0x47C`, `*_HIGH`, `0x41C`/`0x420`/`0x424`/`0x458`/`0x45C`, `0x000`, `0x088`, `0x008`, 미매핑	§2.1, §4.8, §9.2	`0x450` 읽기 금지. 끝나면 반드시 원복
T-03	descriptor 1장으로 복사하기 (`REG-007`, `REG-015`, `DESC-001` ~ `DESC-003`, `DESC-007`, `FUNC-001`, `FUNC-009`, `AXI-001`)	`0x47C`, `0x428`, `0x454` (참고)	§4.6, §5, §7.2	`DESCRIPTOR_ID` = `0xA0`은 참고 체크

테스트	제목	주로 쓰는 레지스터	근거 절	이 문서 기준 주의
T-04	체인 따라가기와 모으기(gather) (`DESC-004`, `DESC-009`, `FUNC-002`)	`0x404`, `0x454` (참고)	§4.4.1, §5, §6.2	체인 하나 = `TRANSFER_ID` +1
T-05	나눠 쓰기(scatter) (`FUNC-003`)	— (데이터 경로)	§5	—
T-06	체인 길이를 1장부터 32장까지 (`FUNC-004`)	`0x454` (참고)	§5	—
T-07	1바이트 단위 길이 전송 (`FUNC-005` ~ `FUNC-007`, `AXI-002`, `AXI-003`)	descriptor `x_len`	§1.2, §4.4.6, §5	`x_len` = 바이트 수 − 1
T-08	중간 알림(EOT)과 descriptor 번호 (`DESC-005`, `DESC-006`, `REG-016`, `IRQ-001`)	`0x084`, `0x454`	§4.6.2, §6.2	알림 때 읽은 `DESCRIPTOR_ID`는 참고 체크 (다음 장 id가 보일 수 있음)
T-09	알림 막기(MASK)와 지우기(W1C) (`IRQ-002` ~ `IRQ-005`)	`0x080`, `0x084`, `0x088`, `0x428`	§4.2, §6.4	완료는 `TRANSFER_DONE` 반복 읽기. `0x0`에서 첫 `irq`는 접수
T-10	출발 버튼 규칙과 연속 예약 (`REG-009` ~ `REG-014`, `FUNC-008`, `IRQ-006`)	`0x408`, `0x404`, `0x42C`, `0x428`, `0x47C`	§4.4.1, §4.4.2, §4.5.1, §7.3	중복 요청 시험은 1일 때만 판정
T-11	전송 중 끄기(ENABLE=0)와 다시 켜기 (`REG-006`, `REG-024`)	`0x400`, 추적 레지스터, 설정 레지스터, `0x43C` (관찰)	§4.3.1, §8, §5 "hang 없음"	§8 `ENABLE=0` 열을 처음 측정. `DBG_STATUS`는 관찰 기록
T-12	일시정지(PAUSE)와 다시 시작 (`REG-008`, `REG-018`)	`0x400`, `0x448`, `0x43C` (관찰)	§4.5.4, §8	§8 `PAUSE=1` 열을 처음 측정
T-13	주소 정렬과 4 KB 경계 (`AXI-004`, `DESC-008`, `FUNC-010`)	`0x47C`, descriptor 주소 필드	§4.4.4, §4.6.1, §5	descriptor 4 KiB 걸침은 관측 후 확정 (64 B 정렬 권장)
T-14	느린 버스(backpressure)와 AXI 속성 (`AXI-005`, `AXI-007`)	— (버스 신호 `AxCACHE`/`AxPROT`를 고정 기대값과 비교. `0x014` 값은 T-01이 봄)	§4.1.6	레지스터 값 ≠ 버스 출력
T-15	에러 응답과 잘못된 descriptor (`ERR-001` ~ `ERR-004`)	`0x400`, `0x088`, `0x428`	§4.6.3, §5	판정은 "hang 없음"만. 길이 0 descriptor는 만들 수 없음

SOR v2.0.0에서 제외된 요구사항과 이 문서의 관계

원래 ID	내용	이 문서
`REG-017`	`CURRENT_*_ADDRESS`가 진행에 따라 바뀜	디버그 레지스터 규칙(§4.7) 대상이라 제외. 레지스터 설명은 §4.7.1·§4.7.2에 그대로 둠
`AXI-006`	응답 안 온 요청 여러 개 동시 처리	레지스터 맵과 무관
`COV-001`	toggle coverage 측정	레지스터 맵과 무관

10. ADDR_WIDTH = 64 전환 시 영향

본 문서는 **`DMA_AXI_ADDR_WIDTH = 32` 확정**(결정 2) 기준입니다. 향후 64로 전환할 경우를 대비해 영향 범위를 분리해 둡니다.

항목	`= 32` (확정)	`= 64`
`HAS_ADDR_HIGH`	0	1
`0x490` `DEST_ADDRESS_HIGH`	**Inactive**, 읽기 0	**RW**, 목적지 주소 `[63:32]`
`0x494` `SRC_ADDRESS_HIGH`	**Inactive**, 읽기 0	**RW**, 소스 주소 `[63:32]`
`0x4BC` `SG_ADDRESS_HIGH`	**Inactive**, 읽기 0	**RW**, descriptor 주소 `[63:32]`
`0x498` / `0x49C` 디버그 HIGH	읽기 0	현재 주소 `[63:32]`
descriptor `dest/src/next` 상위 32비트	무시	유효
래퍼 포트 폭	32	64

전환 시 개정 대상

대상	내용
본 문서	§3 (HIGH 5건의 접근·상태), §4.8, §9.2, 부록 A
Sanity 항목 정의서 (`SGDMA-SANITY-rel-v2.0.0`)	`SAN-007` 체크 4건 (HIGH 레지스터 기대값)
`tb_sg_dma.sv`	주소 폭 localparam, `SAN-007`의 HIGH 체크
`sgdma_top.v`	`DMA_AXI_ADDR_WIDTH` + 3개 마스터 포트의 주소 폭
SOR (`SGDMA-SOR-rel-v2.0.0`)	**T-02** 통과 기준의 `0x490`/`0x494`/`0x4BC` 행 (원래 `REG-021` 서술), **T-07** (원래 `AXI-003`), 2.5절 공통 메모리 영역

판단 기준: SoC 통합 시 주소 공간이 **4 GB를 넘는지**. 넘지 않으면 32 유지가 맞습니다.

---

11. 미확정 항목

결정 7건이 반영되어 미확정 항목은 5건 → **3건**으로 줄었습니다. v2.0.0에서 새로 해소된 항목은 없습니다.

#	항목	내용	해소 방법	담당
1	**`AXI_AXCACHE` 성능 영향**	IP DV는 `4'b0000` 확정(결정 1). 다만 MM→MM 목적지가 Normal memory라면 `4'b0011`이 통상값이며, `0000`은 interconnect가 write를 직렬화할 수 있음	실제 interconnect에서 대역폭 측정 후 재결정	**SoC DV 이관**
2	SG 모드 시 주소/길이 레지스터 무시 여부	`0x410`/`0x414`/`0x418`에 값을 써도 descriptor가 이기는지 — RTL상으로는 mux로 descriptor가 선택되나 미측정	FT 시나리오로 실측. **SOR v2.0.0 15개 테스트에는 이 시험이 없음** — 필요하면 T-03 변형(레지스터에 다른 값을 써 둔 채 SG 전송)으로 추가	IP DV
3	`0x430` `STATUS` 용도	상시 0. 예약인지 폐기된 레지스터인지 불명	ADI 공식 레지스터 문서 대조	IP DV (낮음)

해소된 항목 (상세는 부록 D)

이전 #	항목	해소
1	`AXI_AXCACHE` 명시 여부	**결정 1** — `4'b0000`으로 래퍼에 명시. 성능 재검토만 SoC DV로 이관
2	`DMA_AXI_ADDR_WIDTH` 32 vs 64	**결정 2** — 32 확정

이전 #	항목	해소
4	`DBG_IDS0/1` src 측 필드 전진	**판정 대상에서 제외**로 정리 (§ 4.7.4). Gray 인코딩 특성상 안정적 기대값을 정의할 수 없으므로 미확정으로 둘 항목이 아님

> **참고 — SOR v2.0.0의 "관측 후 확정" 항목.** 아래는 이 문서의 레지스터 사양이 아니라 **동작**이 아직 정해지지 않은 것들입니다. SOR 쪽에서 관측해 정답을 정합니다. descriptor 4 KiB 경계 걸침(T-13), 정렬 안 된 데이터 주소(T-13), 에러 응답 처리(T-15), `ENABLE=0` 시 남은 descriptor 처리(T-11), 알림 시점의 `DESCRIPTOR_ID`(T-08), 체인에서 `TRANSFER_PROGRESS` 누적(T-12). 결과가 나오면 해당 절(§ 4.5.4, § 4.6.2, § 4.6.3, § 5)에 반영합니다.

부록 A — 리셋 직후 readback 기대값

HW 리셋 해제 직후, `CONTROL` 쓰기 전 상태입니다. **M = Sanity 실측 확인.**

Offset	이름	기대값	근거
`0x000`	`VERSION`	`0x0004_0565`	**M**
`0x004`	`PERIPHERAL_ID`	`0x0000_0000`	**M**
`0x008`	`SCRATCH`	`0x0000_0000`	**M**
`0x00C`	`IDENTIFICATION`	`0x444D_4143`	**M**
`0x010`	`INTERFACE_DESCRIPTION_1`	`0x3807_0303`	**M**
`0x014`	`INTERFACE_DESCRIPTION_2`	`0x0000_0000`	**M**
`0x080`	`IRQ_MASK`	**`0x0000_0003`**	**M**
`0x084`	`IRQ_PENDING`	`0x0000_0000`	**M**
`0x088`	`IRQ_SOURCE`	`0x0000_0000`	**M**
`0x400`	`CONTROL`	`0x0000_0000`	**M**
`0x404`	`TRANSFER_ID`	`0x0000_0000`	**M**
`0x408`	`TRANSFER_SUBMIT`	`0x0000_0000`	R
`0x40C`	`FLAGS`	**`0x0000_0002`**	**M**
`0x410`	`DEST_ADDRESS`	`0x0000_0000`	**M**
`0x414`	`SRC_ADDRESS`	`0x0000_0000`	**M**
`0x418`	`X_LENGTH`	`0x0000_0000`	**R** ⚠
`0x41C`	`Y_LENGTH`	`0x0000_0000`	**M**
`0x420` / `0x424`	`*_STRIDE`	`0x0000_0000`	R
`0x428`	`TRANSFER_DONE`	`0x0000_0000`	**M**
`0x42C`	`ACTIVE_TRANSFER_ID`	`0x0000_0000`	**R**
`0x430`	`STATUS`	`0x0000_0000`	R
`0x434` ~ `0x444`	디버그군	**비교 제외**	—
`0x448`	`TRANSFER_PROGRESS`	`0x0000_0000`	R
`0x44C` / `0x450`	partial	`0x0000_0000`	R
`0x454`	`DESCRIPTOR_ID`	`0x0000_0000`	R
`0x458`	`FRAMELOCK_CONFIG`	`0x0000_0000`	**M**
`0x45C`	`FRAMELOCK_STRIDE`	`0x0000_0000`	R
`0x47C`	`SG_ADDRESS`	`0x0000_0000`	**M**

Offset	이름	기대값	근거
`0x490` / `0x494` / `0x4BC`	주소 `*_HIGH`	`0x0000_0000`	**M**
`0x498` / `0x49C`	디버그 `*_HIGH`	`0x0000_0000`	R
미매핑	—	`0x0000_0000`	**M**

> ⚠️ **`0x40C` = `0x2` 와 `0x080` = `0x3` 이 유일한 비영(非零) 리셋값입니다.** 나머지는 모두 0이므로, "전 레지스터 0" 가정 테스트는 **이 둘에서만** FAIL합니다. 처음 이 IP를 다루는 사람이 가장 많이 걸리는 지점입니다.

> ⚠ **`0x418` `X_LENGTH` 는 upstream 문서가 `7` 이라고 적는 값입니다.** 본 구성 (`DMA_LENGTH_ALIGN = 0`)에서는 RTL 분석상 `0` 이지만 **측정되지 않았습니다.** 이 표에서 가장 먼저 확인할 항목입니다. (SOR v2.0.0에도 이 값을 재는 테스트는 없습니다.)

> ⚠️ **`0x450` 은 이 표의 값을 확인하기 위해서라도 읽지 마십시오** — §4.5.5. 전수 sweep 시나리오를 만들 때 이 주소를 빼는 것이 설계 조건입니다.

> ⚠️ **디버그군(`0x434` ~ `0x444`)은 이 표에서 비교하지 않습니다** — §4.7 규칙.

부록 B — Gray 코드 변환표 (ID_WIDTH = 4)

`DBG_IDS0/1` 해석용입니다(관찰용 — §4.7). `b2g(n) = n ^ (n >> 1)`.

Gray 코드는 **이웃한 숫자끼리 비트가 하나만 다른** 표기법입니다. 그래서 값을 그대로 빼면 거리가 나오지 않고, 먼저 binary로 바꿔야 합니다.

binary	Gray	binary	Gray
0	`0x0`	8	`0xC`
1	`0x1`	9	`0xD`
2	`0x3`	10	`0xF`
3	`0x2`	11	`0xE`
4	`0x6`	12	`0xA`
5	`0x7`	13	`0xB`
6	`0x5`	14	`0x9`
7	`0x4`	15	`0x8`

사용 예: `ENABLE=1` 이후 burst 5개가 처리되고 정지한 상태라면 dest 네 필드가 모두 `0x7` → `DBG_IDS0 = 0x0707_0707`.

거리 계산: 두 필드의 in-flight burst 수 = `(g2b(x) − g2b(y)) mod 16`. **Gray 값을 그대로 빼면 틀립니다.**

부록 C — RTL 소스 위치

`rtl/third_party/adi_hdl/library/axi_dmac/`

파일	본 문서에서 인용한 내용
`axi_dmac_regmap.v`	`0x000` ~ `0x088`, `CONTROL`, 디버그 레지스터
`axi_dmac_regmap_request.v`	`0x404` ~ `0x47C`, SG generate 블록, 0-replication 합법 사례
`dmac_sg.v`	descriptor fetch, 완료 추적 FIFO, `DESCRIPTOR_ID`

파일	본 문서에서 인용한 내용
`axi_dmac.v`	파라미터 전달 (`AXI_AXCACHE` 기본값 경로)
`axi_dmac_transfer.v`	전송 경로 구조
`axi_dmac_reset_manager.v`	`DBG_STATUS` 상태 머신
`axi_dmac_burst_memory.v`	**0-replication 패치 지점 (line 163)**

패치 현황: `axi_dmac_burst_memory.v:163` 0-replication 1건. 그 외 ADI 원본 무수정.

> SOR v2.0.0 T-15는 에러 응답 처리를 보기 위해 `response_handler.v`, `axi_dmac_response_manager.v`, `src_axi_mm.v`, `dest_axi_mm.v`를 먼저 읽도록 정했습니다. 이 파일들은 레지스터 맵과 직접 관련이 없어 위 표에는 넣지 않았습니다.

부록 D — 결정 및 충돌 해소 이력

D.1 확정된 설계 결정 (2026-10-06)

#	항목	제안 (hrcho)	결정	레지스터 맵 영향
1	`AXI_AXCACHE`	미지정 상태. `4'b0011` 명시 검토 요청	**`4'b0000` 유지 + 명시**	없음 (실측값과 동일)
2	`DMA_AXI_ADDR_WIDTH`	as-built 32 vs rev.001 가정 64	**32 확정**	없음 (as-built 유지)
3	모듈명	`sgdma_top`으로 통일	**동의**	없음
4	래퍼 파라미터 전부 명시 (결정 당시 36개, 지금 38개)	기본값과 같아도 의도를 코드에 남김	**동의**	없음. §1.1에서 "미지정" 행 소멸
5	AXI ID/LOCK 포트 노출	이미 반영, Sanity 사이클 단위 동일 확인	**동의**	없음
6	레지스터 명칭 4건 upstream 통일	D.2 ⑤~⑧	**동의**	명칭만. **TB 반영 1건** → 부록 E
7	Sanity 보강 2건	`X_LENGTH`·`ACTIVE_TRANSFER_ID` 리셋값	**재조정** → D.3	없음 (46 유지)

> **결정 1·2가 "값을 바꾸지 않는 결정"인 것이 중요합니다.** 기존 46개 체크의 기대값이 그대로이므로 **재측정 없이 문서를 확정**할 수 있었습니다. 만약 `AXCACHE`를 `4'b0011`로 바꿨다면 `INTERFACE_DESCRIPTION_2` 기대값이 `0x0000_0030`으로 변하고 `SAN-001` check 5를 수정해야 했습니다.

D.2 통합 시 해소된 충돌 (2026-10-02)

선행 문서 2건(jsha rev.001 / hrcho v0.2)이 어긋났던 지점입니다. **보존 목적으로 남깁니다** — 같은 종류의 오류가 재발했을 때 참조점이 됩니다.

#	항목	rev.001 (jsha)	v0.2 (hrcho)	확정	근거
①	`INTERFACE_DESCRIPTION_2` 리셋값	`0x0000_0030`	`0x0000_0030`	**`0x0000_0000`**	**실측** — 양쪽 다 regmap 로컬 기본값을 본 오류
②	`DMA_AXI_ADDR_WIDTH`	64 (기본값 가정)	32	**32**	**as-built** + 결정 2

#	항목	rev.001 (jsha)	v0.2 (hrcho)	확정	근거
③	`DESCRIPTOR_ID` 완료 후 값	"stale 값이 보일 수 있음"	기대값 명시 없음	**마지막 완료 descriptor id 유지** (계약 아님)	**실측** — 3-descriptor 체인 `0xB2`
④	`TRANSFER_SUBMIT` 접근 유형	`RW1S-SC` + polling 규칙	`RW`	**`RW1S-SC`**	rev.001이 정확
⑤	`0x448` 명칭	`TRANSFER_PROGRESS`	`TRANSFER_LENGTH`	**`TRANSFER_PROGRESS`**	완료 시 0으로 지워지는 동작에 부합
⑥	`0x014` 명칭	`INTERFACE_DESCRIPTION_2`	`AXI_CONFIG`	**`INTERFACE_DESCRIPTION_2`**	`0x010`과 짝, upstream 명칭
⑦	`0x454` 명칭	`DESCRIPTOR_ID`	`SG_DESCRIPTOR_ID`	**`DESCRIPTOR_ID`**	upstream 명칭
⑧	`0x434`/`0x438` 명칭	`CURRENT_*_ADDRESS`	`DBG_*_ADDR`	**`CURRENT_*_ADDRESS`**	upstream 명칭

> **①이 이 문서의 M/R/D 등급 체계를 만든 사건입니다.** 서로 독립적으로 RTL을 읽은 두 사람이 **같은 틀린 값**에 도달했습니다. 정적 분석의 교차 검증으로는 걸러지지 않는 오류이며, 돌려보기 전까지는 알 수 없었습니다.

D.3 결정 7의 재조정

원안 (2026-10-06 오전): `X_LENGTH`(0x418)와 `ACTIVE_TRANSFER_ID`(0x42C)의 리셋값 체크를 `SAN-003`에 추가 (46 → 48 checks). 팀 동의를 받았습니다.

재조정 (2026-10-06 오후, 후속 논의): **Sanity는 현행 46 checks 범위를 유지**합니다. 이어서 SOR에서도 **`RST` 요구사항 6건을 삭제**하고 3-1을 "해당 사항 없음"으로 정리했으므로, **미측정 리셋값은 Sanity에도 SOR에도 없습니다.** (SOR v2.0.0도 이 결정을 유지합니다.)

	내용
판단	Sanity의 역할은 **회귀 게이트**이지 커버리지 도구가 아니다
범위	리셋값 검증은 현 단계에서 **투자 대비 우선순위가 낮다**고 판단해 Sanity·SOR 양쪽에서 제외한다
이유	2건만 추가해도 나머지 10여 건이 남는다. 전부 닫을 계획이 없다면 2건 추가는 완결성을 주지 못한다
기록	미측정 항목과 사유, 나중에 측정할 때의 제약은 §9.3에 보존한다

남는 상태: `X_LENGTH` 리셋값은 **upstream 문서(`7`)와 RTL 분석(`0`)이 어긋난 채 미측정**이며, **수행 계획 없이 인계**됩니다. 근거 등급 **R**로 표기되어 있고 **측정 전까지 확정된 값이 아닙니다.** 완벽하게 할 수 없는 것을 명시하고 넘어가는 것이 이 재조정의 전제입니다.

> **결정을 조용히 흘리지 않고 기록으로 남기는 이유.** 합의된 항목이 실행되지 않으면, 나중에 문서만 보는 사람에게는 "합의해놓고 안 했다"로 보입니다. 범위가 바뀐 것이라면 바뀐 이유가 같이 남아야 합니다.

D.4 v2.0.0 문서 정정 (2026-10-16)

SOR 축소안 5개를 리뷰하는 과정에서 **같은 오해가 여러 후보에서 반복**되었습니다(`SGDMA-SOR-INTEGRATION-v1.0` 2.2절). 같은 실수가 반복된다는 것은 원본 문서에 오해를 부르는 부분이 있다는 뜻이므로, 이 문서도 함께 고쳤습니다. **RTL 사실(값)은 바뀌지 않았습니다.**

#	오해	나온 후보	이 문서의 정정	위치
1	"길이 0 descriptor = `x_len 0xFFFFFF`"	5/5	`0xFFFFFF`는 16 MiB. 길이 0은 만들 수 없음	§4.4.6, §5
2	`DBG_STATUS`를 합격 기준으로 씀	5/5	디버그 레지스터는 관찰 기록만. 규칙을 §4.7 서두에 모음	§4.7, §4.7.3
3	자기 참조 체인을 "시간 안에 끝나야 hang 아님"으로 판정	5/5	"hang 없음" = `ENABLE=0` 후 버스 idle + 다시 켜서 정상 전송	§5
5	완료 후 `DESCRIPTOR_ID`를 합격 기준으로 씀	4/5	구현 특성(계약 아님). 알림 시점에는 다음 장 id가 보일 수 있음	§4.6.2
6	마스크된 설정에서 `irq`를 기다림 / `0x0`에서 첫 `irq`를 완료로 봄	4/5	완료는 `TRANSFER_DONE` 반복 읽기. `0x0`에서 첫 알림은 접수	§4.2.1, §6.3, §6.4, §7.2

(번호는 통합 문서 2.2절의 번호입니다. #4, #7, #8은 testbench·시험 절차의 문제라 SOR에서만 다룹니다.)

부록 E — 코드 반영 현황

결정 7건의 코드 반영은 2026-10-06에 완료되었습니다. 각 단계를 별도 커밋으로 나누고 매 단계마다 Sanity를 돌려, 어느 변경이 무엇을 바꿨는지 분리할 수 있게 했습니다. (v2.0.0은 문서 개정이며 코드 변경은 없습니다.)

E.1 적용 순서와 결과

단계	변경	대상 파일	결과
1	모듈명 `sg_dma_top` → `sgdma_top`	`sgdma_top.v`, `rtl.f`, `tb_sg_dma.sv`	**46/46**
2	`AXI_AXCACHE = 4'b0000`, `AXI_AXPROT = 3'b000` 명시	`sgdma_top.v`	**46/46 + 타임스탬프 19개 동일**
3	레지스터 명칭 upstream 통일 + 주석 정리	`tb_sg_dma.sv`	**46/46**

2단계가 실질적인 검증이었습니다. 명시 전에는 `axi_dmac`의 암묵적 기본값이 적용되고 있었고 실측값이 `0x0000_0000`이었습니다. 명시적으로 `4'b0000`을 넘겼을 때 **레지스터 값도 AXI 트랜잭션 타임스탬프 19개도 전부 그대로**였으므로, §4.1.6의 분석이 옳았음이 확인되었습니다. 값이 바뀌었다면 전제가 틀렸다는 뜻이었을 것입니다.

E.2 레지스터 명칭 — 실제 반영 범위

합의는 명칭 4건이었으나, 그중 TB에 실존한 것은 2건이었습니다.

부록 D	레지스터	TB 존재	조치
⑤	`0x448 TRANSFER_PROGRESS`	없음	조치 불필요
⑥	`0x014 INTERFACE_DESCRIPTION_2`	**있음** (`REG_AXI_CONFIG`)	**변경 완료**
⑦	`0x454 DESCRIPTOR_ID`	**체크 문자열에 `SG_DESCRIPTOR_ID`**	**변경 완료**
⑧	`0x434`/`0x438 CURRENT_*_ADDRESS`	없음	조치 불필요

> **⑦은 로그를 보고서야 발견했습니다.** localparam은 `REG_DESCRIPTOR_ID`로 올바랐지만 `SAN-006`의 **출력 문자열**이 `SG_DESCRIPTOR_ID`였습니다. 같은 레지스터(`0x454`)가 한 TB 안에서 `SAN-005`는 `DESCRIPTOR_ID`, `SAN-006`은 `SG_DESCRIPTOR_ID`로 찍히고 있었습니다. **소스만 보고 판단했으면 놓쳤을 항목**입니다.

변경된 식별자와 문자열:

분류	변경 전	변경 후
localparam	`REG_IF_DESCRIPTION` / `EXP_IF_DESCRIPTION`	`REG_INTERFACE_DESCRIPTION_1` / `EXP_INTERFACE_DESCRIPTION_1`
localparam	`REG_AXI_CONFIG` / `EXP_AXI_CONFIG`	`REG_INTERFACE_DESCRIPTION_2` / `EXP_INTERFACE_DESCRIPTION_2`
출력 문자열	`"INTERFACE_DESCRIPTION"`	`"INTERFACE_DESCRIPTION_1"`
출력 문자열	`"AXI_CONFIG"`	`"INTERFACE_DESCRIPTION_2"`
출력 문자열	`"SG_DESCRIPTOR_ID"`	`"DESCRIPTOR_ID"`

E.3 TB에 남긴 경고 주석

같은 오류가 반복되지 않도록 오답의 출처를 코드에 박아두었습니다.

```
        // AXI_AXCACHE = 4'b0000 (Device Non-bufferable), AXI_AXPROT = 3'b000.
        // Explicitly specified in sgdma_top as of 2026-10-06 (team decision); before
        // that they relied on axi_dmac's own defaults and measured the same value.
        // NOTE: axi_dmac_regmap declares a LOCAL default of 4'b0011 - do NOT compute
        // the expected value from that file.  Two independent reviews did exactly
        // that, both got 0x30, and both were wrong.
```

`SAN-003`에도 커버 범위를 명시했습니다. SOR의 Reset / Clock 항목이 Sanity에 위임되므로, **이 TB 자체가 리셋 검증의 근거물**이 되기 때문입니다.

```
        //   Covers 10 of the ~30 registers listed in SFR map appendix A.  The rest
        //   are intentionally left to the verification team.  Do not read this TC as
        //   proof that every reset value has been checked.
```

`SAN-007`에는 `0x450` 금지 사유를 남겼습니다.

```
        //   0x450 (PARTIAL_TRANSFER_ID) is deliberately NOT read here - a read pops
        //   the internal FIFO.  See SFR map 4.5.5.
```

(TB 주석이 가리키는 "SFR map appendix A", "SFR map 4.5.5"는 v2.0.0에서도 같은 번호입니다.)

E.4 부수적으로 정정된 것

항목	내용
TB 헤더 체크 개수	`SAN-005`가 `(5 checks)`로 적혀 있었으나 **실제 6개**. 합계 줄도 추가
`EXP_PERIPHERAL_ID` 주석	"`ID`를 override하지 않는다"는 **틀린 설명** → 래퍼가 기본값 0을 넘긴다로 정정
`SAN-006` 참조	`SFR map 3.7` → **`4.2.2`**
한글 작업 주석 2곳	영문 설명으로 교체
버전 문자열	`SGDMA-SANITY-v1.0`, `SGDMA-SFR-v1.0`

개정 이력

버전	일자	작성	내용
rev.001	2026-09-30	jsha	RTL 정적 분석 기반 전체 사양
v0.2	2026-10-01	hrcho	실측 기반 요약본

버전	일자	작성	내용
rev.002	2026-10-02	hrcho	통합 — rev.001 구조 + 실측값, 근거 등급(M/R/D) 신설, Sanity 연계·ADDR_WIDTH 전환 영향 신설, 충돌 8건 해소
rel-v1.0.0	2026-10-06	hrcho	**검증팀 인계본** — 설계 결정 7건 반영(결정 7은 재조정), 미확정 5건 → 3건, 모듈명 `sgdma_top` 통일, 래퍼 파라미터 36 → 38, Sanity 미커버 영역 명시(§9.3), 코드 반영 현황(부록 E)·결정 재조정 기록(부록 D.3) 신설
rel-v2.0.0	**2026-10-16**	—	**SOR rel-v2.0.0(테스트 15개)·SANITY rel-v2.0.0에 맞춘 문서 개정. RTL 사실(주소·비트·접근·리셋값·근거 등급·파라미터·Gray 표·RTL 위치·결정 이력)과 절 번호는 그대로.** ① `x_len` / `X_LENGTH` "바이트 수 − 1", 길이 0 불가, v1 SOR ET-005 정정 메모(§4.4.6, §5) ② 자기 참조 체인에 SOR 2.3절 "hang 없음" 정의 연결(§5) ③ 디버그 레지스터 관찰 전용 규칙을 §4.7 서두로 모음, `DBG_STATUS` "판정" 표현 삭제 ④ 요구사항 ID를 T-01 ~ T-15 기준으로 갱신(원래 ID 괄호 유지), §9.3을 SANITY v2 5장과 맞춤, §9.4 신설 ⑤ `IRQ_MASK = 0x0` 접수 알림 주의(§4.2.1, §6.3, §6.4 신설, §7.2) ⑥ `DESCRIPTOR_ID` 알림 시점 값·구현 특성(§4.6.2) ⑦ §8이 RTL 분석(미측정)이며 T-11/T-12에서 처음 측정 ⑧ 부록 D.4 신설 ⑨ 처음 보는 사람용으로 다시 씀(비유·용어 풀이·한 줄 요약·⚠️ 주의). 근거: `SGDMA-SOR-INTEGRATION-v1.0` 7장 #4 ~ #8

다음 개정 예정: 본 문서의 R 항목 중 **리셋값**은 현재 수행 계획이 없습니다. 향후 측정하게 되면 §9.3의 항목들이 M으로 승급하며, 특히 **`X_LENGTH` 가 `0` 인지 `7` 인지**가 확정되면 §4.4.6과 부록 A를 갱신해야 합니다. SOR v2.0.0의 T-11/T-12가 실행되면 §8의 `ENABLE=0` · `PAUSE=1` 열을, T-08·T-13·T-15의 "관측 후 확정" 결과가 나오면 해당 절을 갱신합니다.